

products, in the form of packages. Why have packages not taken off despite India's low labour cost advantages? Because, notwithstanding the low cost at which Indian companies can develop such packages, there are serious barriers to entry into this market which are worse than those for software services. Indian firms and their developers are not sufficiently familiar with the foreign package markets they seek to penetrate, and their distance from those markets makes it hard to keep up with changing needs and standards. The Indian domestic market is a poor guide for software developers thanks to differences in user needs, in work and hardware environments, and thanks to the generally low level of innovation.

2. Uneven Export Destination: US Domination

Indian companies have exported software to more than forty countries but there is a heavy reliance on the US market, as the table below indicates.

Table 7: Breakdown of Indian Software Exports by Destination (1997/98)

<i>Destination</i>	<i>Proportion of Total Exports</i>
US	65%
UK	10%
Other Europe	10%
Japan	5%
Other	10%

Source: Dataquest (1998)

The US market dominates Indian software exports partly because it is by far the world's largest software market, constituting around half of all software sales in the 1990s, and partly because American information technology and financial services companies have moved much more quickly than their European counterparts to take advantage of offshore programming. Until recently (till the 11/9 tragedy) the US has also had more liberal immigration rules for work or residence than most other developed countries. However, India is also more 'locked-in' to the US market than others because many Indian businesses have links through family members or friends who are US residents; because many software developers are US-trained and so understand that market best; and because there is a vast predominance of US firms in the all-important collaborations which provide so much of India's software export market.

The US-orientation of exports might present a limitation to future growth because US share of the world market is slowly declining. However, the American market is still expanding substantially in absolute terms plus, since the early 1990s, Indian software exports have shown

their ability to grow in other markets, such as those of Asia and Europe.

3. Uneven Divisions of Labour: 'Body Shopping Trap'

[A] Uneven Locational Divisions: Onsite and Offshore Work

Much of India's export work developing custom software is actually carried out at the client's site overseas ('onsite') rather than offshore in India. Back in 1988, an average of 65% of export contracts were carried out wholly at the client site, while 35% contained some offshore elements. This translated into just under 75% of Indian software export development taking place overseas and only 25% in India. This was even true of work in India's export processing zones, which were intended to be bases for offshore work. Subsequent surveys have shown that the amount of work carried out offshore has increased within individual firms. The trend is particularly noticeable in the subsidiaries of multinational firms. However, a significant amount of onsite work has remained within the industry overall so that, by 1997/98, it was still true that more than half of all software services export earnings came from onsite work (Dataquest 1998). This forms the basis for an international locational division of labour within India's software export trade.

[B] Uneven Skill Divisions: Dominance of Programming

During the period 1988-1998, at least 65% of export contracts were solely for programming work billed on a 'time and materials' basis, with programming figuring strongly in the remaining one-third of contracts (Dataquest 1998). As with offshore work, there have been changes within individual client-contractor relationships, but there has been much less change within overall industry averages. So, in general terms, India's software export trade has been characterized by an international skill division of labour such that the majority of software contracts allocate only the less-skilled coding and testing stages to Indian workers. That is to say, Indian workers have far more often been used as programmers, working to requirements and design specifications set by foreign software developers, rather than as systems analysts or designers.

4. Uneven Market Share: Economic Concentration of Production

Since Indian software exports are services rather than goods, examples cannot easily be displayed to potential buyers to establish credibility. There is therefore a heavy reliance on reputation, track record, references and the skills and appearance of the marketing team, which all go together to determine the Indian firm's credibility. All these credibility-related factors, which principally hinge on track record and

spending on marketing, obviously work to the advantage of larger, longer-established firms. As an example, a few firms have had to invest in export marketing for two or three years before getting their first export order.

5. Uneven Site Distribution: Locational Concentration of Production

Software companies are not distributed evenly throughout India, but are mainly located around a few major Indian cities, especially around Bangalore. The table below indicates the location of company headquarters for over 550 software companies (including a number focused on the domestic market).

Table 8: Location of Indian Software Company Headquarters

City	Number of Software Firm HQs
Bangalore	152
Mumbai (Bombay)	122
Chennai (Madras)	93
Delhi/New Delhi	86
Hyderabad/Secunderabad	34
Calcutta	27
Pune	22
Other	22
Total	558

Source: Dataquest (1998)

There are several factors behind this unequal siting growth of software development centers in India. (Such as skills availability, proximity to customers, quality of life, infrastructure etc).

B.4.2 People Management Problems in the Indian Software Industry

There is widespread dissatisfaction among senior managers in the software industry with the performance of software projects. This takes the form of inordinate delays, time and cost overruns, user dissatisfaction and maintenance problems. "High Burn Out" is a common problem in most Indian software companies (we discussed this phenomenon in Chapter 11). In order to overcome these problems, software companies are investing heavily to make improvements in their software processes and practices. Many of them have realized that lasting improvements require significant changes in the way they manage, develop and use people. In many software companies the work of managing people is seen as a responsibility of the Human Resources Department. The practices relating to management of people are done in an ad-hoc manner with little realization of their impact on the motivation and morale of staff. The reluctance of project leaders/managers to undertake such responsibilities is because they have not been trained to perform func-

tions relating to management of people and also the fact that they are constantly under the pressure of delivery deadlines.

B.4.3 Infrastructure related Problems

Infrastructure consists of transport, electricity, and connectivity; land availability, quality of life, government facilities etc. In this section, we discuss issues that are considered as deterrent to foreign investors looking to India as an outsourcing destination. Seen from the eyes of a foreign investor, the perception is that at present, there are severe limitations in general and infrastructure are a primary weakness of the Indian software industry.

Good communication infrastructure is considered vital for the continued growth of the software industry. Overall, the data communication infrastructure in India is expensive and in limited supply. It appears that the problem has a significant institutional component, with government agencies like the department of telecommunication and VSNL, until recently the sole ISP in India, unwilling to give up their stranglehold on telecommunications. Clearly, the poor communication infrastructure has affected the diffusion of the Internet domestically, and through that, has discouraged the growth of new firms that could provide software services for and through the Internet.

As another example of factors that cause negative perception in the minds of foreign investors, consider this—the power grid failures for several hours at a stretch are a common feature even in the metropolitan cities of India. Tap water is contaminated in most Indian locations. The main roads get clogged very often. Traffic snarls are not well-managed. Telephone connections take longer to complete than they would in most other countries. Fully half of the population is illiterate; three-fourths are rural. Village India remains the social and political anchor. There are shooting civil wars in progress in several states owing to ethnic hostilities. The Indian press is not considered friendly and developed.

There is a silver lining though—foreign investors feel that in spite of its problems, India is quietly becoming an international software giant. The Indian government appears to have taken bold steps towards providing the right combination of incentives along with a relatively low hassle support (a radical change in its policies of yesteryears.) What has evolved is something like a coordinated, rational strategy for industrial policy. The preferred strategy does not seem to be “off-shore” development (i.e., lots of cheaper body shops) but the establishment of many highly capable software product organizations that can develop and support complex systems. A concern for quality and desire to make effective use of state of the art technologies is evident. It seems the lessons of software engineering have been well studied by the Indians.

B.4.4 Growing competition from other countries

Currently India boasts more than 300,000 skilled workers work in the software industry. This makes India the second largest English speaking pool of technical manpower in the world. But India should not take a complacent view towards this, as this position may not last for too long. The global reach for software products and services from India is constantly under threat from global competition from other countries that can offer educated manpower at competitive rates. China and Russia are good examples of potential threats.

The table below presents quick comparative figures for India vis-à-vis China. The statistics speak for themselves.

Table 9

India	China	
Per Capita Income	\$ 400	\$ 800
Foreign Direct Investments per year	\$ 3 Billion	\$ 45 Billion
Exports (General)	\$ 45 Billion	\$ 180 Billion
Number of Internet connections	0.8 Million	22 Million
PC Sales in Yr. 2000	1.7 Million	7 Million
Cell Phones (per 1000)	2	34
Software Exports	\$ 6.2 Billion	\$ 0.5 Billion

According to Research Director of Gartner's Asia-Pacific operations, the entry of major Indian software firms will enable China to raise its software exports and within four years it will be able to catch up with India. From last year's \$850 million, it should rise to \$27 billion by 2006. Apart from this, the Indian software industry faces a number of challenges as the labour cost advantages diminish and competition from other countries with supplies of educated and under-utilized workers increases.

Interestingly enough, although competition from other countries such as Philippines and China is typically cited in the press, as the table below shows, most software exporters indicate that their main competitors are located either in the US or in India itself.

Table 10: Location of primary competitors of Indian software firms

Location of Competitors	Number of Firms	% of Firms
India	7.5	82
Israel	12	13
Ireland	12	13
USA	58	63
Singapore	19	21
Philippines	6	7
Eastern Europe/Russia	10	11

Recent trends show that programmers from China, Russia and Romania are proving to be worthy competitors in the low-end segment, for

providing the programming and support services that have been the core of the Indian software exports value proposition.

There are other problems as well; walloped first by the dotcom meltdown and then by the September 11 attacks, India's software industry is now discovering new woes — visa restrictions and police raids overseas that some see as shades of protectionism. Recently, at least two well-known Indian software companies made it in the news media for a negative publicity with incidents of illicit nature taking place abroad. Apart from this, Several recent incidents, including one in which Malaysian police detained 270 Indian information technology workers on suspicion of being illegal workers, have shaken an industry that was looking at steady growth, as overseas firms move software work offshore to benefit from India's low-cost engineers.

REFERENCES

- [1] *The Indian Software Industry* by Ashish Arora, V. S. Arunachalam, Jai Asundi and Ronald and Fernandes—research on the Indian software industry, carried out at Carnegie Mellon University
- [2] *The People Capability Maturity Model—Guidelines for Improving the workforce* by Bill Curtis, William E. Hefely and Sally A. Miller (Pearson Education Low Price Edition)
- [3] *NASSCOM reports* published at various points of time.
- [4] *Sustaining Growth—Indian IT Industry* (Address to the Inaugural Session of NASSCOM 2002 by S Ramadorai Chief Executive Officer at the Tata Consultancy Services)
- [5] *Indian Software Industry: Prospects and Policy* (Shyam Sunder Yale School of Management ASHA, Mid-Hudson Valley and ISA, Pace University, February 21, 2002)
- [6] *The Uneven Profile of Indian Software Exports* by Richard Heeks (1998) Published by Institute for Development Policy and Management—University of Manchester, Precinct Centre, Manchester, M13 9GH, UK

Appendix C

QUALITY RELATED ADDITIONAL TOPICS

C.1 Statistical Quality Control and Statistical Process Control

C.1.1 Introduction

It is important to understand the role of statistics in *Quality Assurance*. *Statistical Process Control* (SPC) and *Statistical Quality Control* (SQC) methodology is one of the most important analytical developments available in this century. Their origins are in the manufacturing industry.

SPC has come to be known as an on-line tool providing close-up views of what's happening to a process at a given moment. SQC provides off-line tools to support analysis and decision making to help determine if a process is stable and predictable. Thus, SPC and SQC are related. When SPC and SQC tools work together, users see the current and long-term picture about processing performance (the "voice of the process"). In sum:

- SPC provides on-line tools that permit a close-up view of what's currently happening to a process.
- SQC provides off-line tools to support analysis and decision-making about process stability over time.

In general, processes achieving the most benefit from SPC and SQC are products with:

- Highly repetitive processes;
- High-volume production and low margins; or
- Narrow tolerances.

To make SPC and SQC work, key parameters indicating product variations are measured and recorded.

SPC and SQC are an effective part of continuously improving a process. When measurements are accurately collected and analyzed, improvements are identified and implemented, and controls established to ensure improvements are permanent; a process is well on its way to meeting quality requirements.

Control Charts are a fundamental tool of SPC and SQC and provide visual representation of how a process varies over time or from unit to unit. *Control limits* statistically separate natural variations from unusual variations. Points falling outside the control limits are considered out-of-control and indicate an unusual source of variation (more about this in section C.1.4).

C.1.2 Process Control

The term *process* (which was discussed in Appendix A) means different things to different people. So first let us give a definition to it (Also see the various definitions given in Appendix A). Especially in the context of a software development or support environment:

A process can be defined as the logical organization of people, materials, energy, equipment, and procedures into work activities designed to produce a specified end result.

Statistical Process Control (SPC) takes advantages of the natural characteristics of any process. All business activities can be described as specific processes with known tolerances and measurable variances. The measurement of these variances and the resulting information provide the basis for continuous process improvement. We'll mention the basic tools of SPC later in this section.

"Controlled processes are" *stable processes* and stable processes enable you to predict results. *Software Process Management (SPM)* is about successfully managing the work processes associated with developing, maintaining and supporting software products and software intensive systems. "Successful management of a process" means that products and services produced by the process conform fully to both internal and external customer requirements and that processes meet the business objectives of the organization responsible for products/services.

This concept of "process management" is based on the principles of *Statistical Process Control (SPC)*. These principles hold that by establishing and sustaining stable levels of *process variability*, processes will yield predictable results. We can then say that the processes are under statistical control.

Software Process Management has four key responsibilities:

1. Define the process
2. Measure the process
3. Control the process (ensure that variability is stable so that results are predictable)
4. Improve the process

C.1.3 Basic Tools of SPC/SQC

These basic tools are: (These are known famously as the "Seven Quality Control Tools")

- Data Tables (for Problem Identification)
- Pareto Analysis (for Problem Identification)
- Cause-and-Effect Diagrams (for both Problem Identification as well as Problem Analysis)
- Trend Analysis (for both Problem Identification as well as Problem Analysis)
- Histograms (for Problem Analysis)
- Scatter Diagrams (for Problem Analysis)
- (Process) Control Charts (for Problem Analysis)

Control Charts, are the most important tool of Statistical Quality Control (SQC). Considering their importance in Process Control, they are discussed separately in the next section.

C.1.4 Control Charts

What are Control Charts?

A control chart is a popular statistical tool for monitoring and improving quality. Originated by Walter Shewhart in 1924 for the manufacturing environment, it was later extended by W. Edward Deming to the quality improvement in all areas of an organization (a philosophy known as Total Quality Management).

The Purpose of Control Charts

The success of Shewhart's approach is based on the idea that no matter how well the process is designed, there exists a certain amount of nature variability in output measurements.

When the variation in process quality is due to "random causes" alone, the process is said to be in-control. If the process variation includes both random and "special causes" of variation, the process is said to be out-of-control. The control chart is supposed to detect the presence of special causes of variation.

In its basic form, the control chart is a plot of some function of process measurements against time. The points that are plotted on the graph are compared to a pair of control limits (The Upper Control Limit and the Lower Control Limit). A point that exceeds the control limits signals an alarm.

An alarm signaled by a control chart may indicate that special causes of variation are present, and some action should be taken, in order to trace and eliminate these causes. On the other hand, an alarm may be a false one, when in practice no change has occurred in the process. The design of control charts is a compromise between the risks of not detecting real changes and of false alarms (This is known as the trade-off between *Type I* and *Type II* error).

Assumptions Underlying Control Charts

The two important assumptions are:

1. The measurement-function (e.g. the mean), that is used to monitor the process parameter, is distributed according to a *normal distribution*. In practice, if your data seem very far from meeting this assumption, try to transform them.
2. Measurements are independent of each other.

Constructing a 3-sigma ("Shewhart-type") control chart

During a stable stage of the process (Section C.1.2 explains what a stable process is):

1. Determine the process parameter that you want to monitor (such as the process mean, or spread).
2. Create the centerline of the plot, according to the target value of your monitored parameter.
3. Group the process measurements into subgroups (samples) by time period or any other suitable parameter. The points to be plotted on the plot, are some function of the process measurements within each subgroup, which estimate the target value.

For example, if you are monitoring your process mean, then the points on the plot should be the sample-means, computed at regular intervals. Denote the point at time t as X_t .

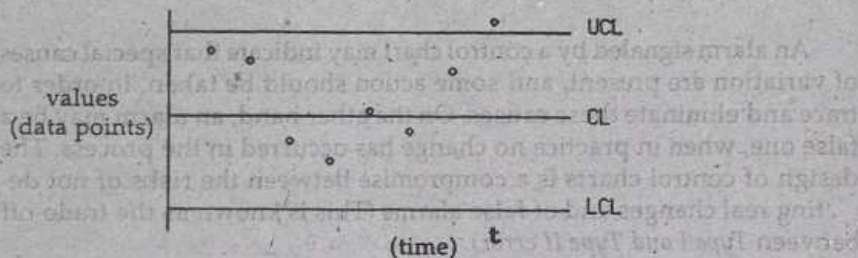
4. Create upper and lower control limits (UCL, LCL) according to the following formula:

$$UCL = CL + 3s$$

$$LCL = CL - 3s$$

where s is the standard deviation of X_t .

For the example above, X_t may be daily means of process measurements. If each daily sample comprises of n measurements, then the standard deviation of X_t is equal to the process standard deviation divided by the root of n .



After the control limits have been set, continue to plot the points on the graph, as a function of time. When a point exceeds either of the control limits (Upper or Lower), it indicates that the process is out of control, and action should be taken (of course, there is a slight chance that is a false alarm).

Analysis of Patterns on Control Charts

A control chart may indicate an out-of-control condition either when one or more points fall beyond the control limits, or when the plotted points exhibit some nonrandom pattern of behavior. The process is "out of control" if any one or more of the criteria is met:

1. One or more points outside of the control limits. This pattern may indicate:
 - A special cause of variance from a material, equipment, method, or measurement system change.
 - Mismeasurement of a part or parts.
 - Miscalculated or misplotted data points.
 - Miscalculated or misplotted control limits.
2. A run of eight points on one side of the centerline. This pattern indicates a shift in the process output from changes in the equipment, methods, or materials or a shift in the measurement system.
3. Two of three consecutive points outside the 2-sigma warning limits but still inside the control limits. This may be the result of a large shift in the process in the equipment, methods, materials, or worker or a shift in the measurement system.
4. Four of five consecutive points beyond the 1-sigma limits.
5. An unusual or nonrandom pattern in the data.
 1. A trend of seven points in a row upward or downward. This may show:
 - Gradual deterioration/wear in equipment/tool.
 - Improvement or deterioration in technique/method.
 2. Cycling of data can indicate:
 - Temperature or other recurring changes in the environment.
 - Differences between operators or operator techniques.
 - Regular rotation of machines.
 - Differences in measuring or testing devices that are being used in order.

6. Several points near a warning or control limit.

Limitations of Control Charts: Sensitizing rules for control charts

The American Standard is based on "three-sigma" control limits (corresponding to 2.7% of false alarms), while the British Standard uses "3.09 sigma" limits (corresponding to 2% of false alarms). In both cases it is assumed that a normal distribution underlies the relevant estimators.

It has been shown that Shewhart-type charts are efficient in detecting medium to large shifts, but are insensitive to small shifts. One attempt to increase the power of Shewhart-type charts is by adding supplementary stopping rules based on runs. The most popular stopping rules were suggested by the "Western Electric Company". These rules supplement the ordinary rule: "One point exceeds the control limits". Here are the most popular Western Electric rules:

- 2 of 3 consecutive points fall outside warning (2-sigma) limits, but within control (3-sigma) limits.
- 4 of 5 consecutive points fall beyond 1-sigma limits, but within control limits.
- 8 consecutive points fall on one side of the centerline.

There are two types of control charts as discussed below.

Variable Control Charts

Variable Control Charts include:

- Individual values
- Median values
- Median
- X-bar
- Range
- Standard deviation
- Cumulative sum (q-sum), and
- Exponentially Weighted Moving Average

Attribute Control Charts

Attribute Control Charts include:

- Percent defective (p-chart)
- Number defective (n-chart)
- Nonconformities per unit (u-chart), and
- Nonconformities (c-chart)

In summary, we can say that as part of the lifecycle of many systems, users need to perform *statistical analysis* of data pertaining to the operation. This function can be required at any point from being part of the initial definition process through post startup performance appraisals. As discussed so far, *Control charts* are a fundamental tool of SPC and SQC and provide visual representation of how a process varies over time or from unit to unit.

C.1.5 Process Performance Variation

Earlier we mentioned about "*special causes*" and "*random causes*". Shewhart categorizes sources of variation as follows:

- Variation due to phenomena that are natural and inherent to the process and whose results are common to all measurements of a given attribute (random variation in the process).
- Variations that have assignable causes (also known as "*special causes*") that could be prevented.

In summary, Shewhart's *Control Charts* (discussion earlier) are techniques for quantifying process behaviour. Control charts and associated methods of statistical quality control (SQC) are most effective when they are used within the broader context of goals you have established and the activities you perform to achieve these goals.

C.2 Software Maintenance Models: The Evolution

C.2.1 Introduction

Software maintenance is recognized as an important part of the software development life cycle (SDLC). The maintenance phase of the software development life cycle is often long and expensive and covers the areas of version upgrade, database migration, language migration, reengineering, functionality upgrades and porting. Our timely and cost-effective maintenance services include critical aspects of planning for post delivery operations, supportability and logistics determination. Maintenance is of three types:

a. *Corrective*

Corrective maintenance involves changing a software application to remove errors and bugs. The three main blocks of corrective maintenance are:

- Design errors
- Logic errors
- Coding errors

Examples may include, simple correction of problems such as misspelled words in the interface of your application and complex design issues relating to incorrect algorithms in your product which at its worst may damage, corrupt or destroy application data.

b. Adaptive

Adaptive software maintenance involves tasks relating to changes in your application's operating environment. These environmental modifications consist mainly of changes to the following:

- Rules, laws, regulations and localizations that affect the application
- New operating systems
- Hardware configuration changes
- Changes in data format
- Changes in supporting utilities

c. Perfective

Perfective maintenance is an activity that we undertake to improve the maintainability, performance or other attributes of your application. Furthermore, perfective maintenance includes all changes, insertions, deletions, modifications, extensions and enhancements made to the application to meet evolving and/or expanding user needs.

C.2.2 History of Software Maintenance Model Evolution

The Maintenance Cycle is related to the Development Cycle. Most readers would be familiar with the different types of models used during the development process. In this section, we intend to provide only a very brief overview of this. The *Waterfall*, *Incremental* and *Evolutionary* models are well known (For details of this, readers can refer to "*Software Engineering: A Practitioner's Approach*" by Roger S. Pressman—McGraw-Hill International). Many people are not aware that there are also models used during the maintenance process. So, the frequently asked questions are: "Is there a maintenance model?" "Is it similar to the development models?" "Are they the same?" "How have they evolved?" This is the reason why we included this topic in this section of this appendix. We now start with a brief discussion of the *history of development models*. This will be followed by a discussion on *history of the maintenance process*.

C.2.1.1 The Code and Fix Development Model

This is one of the early software development models—*Code and Fix*. It simply consisted of two steps: Code and then Fix the code.

This model had many difficulties. There was always a risk of code structure deterioration due to enhancements, updates and a lack of formal change standards. It then became apparent that a *Design Phase* was needed. Often the software did not match the users' needs. This resulted into the feeling that a *Requirements Phase*, too, was required. Eventually, *code quality* became a concern, and so the *Testing Phase* was developed.

C.2.1.2 The Waterfall Development Model

In the Seventies came this model very popularly known as the "Waterfall Model". The Waterfall Model refined Stage wise Model and added two primary enhancements. The first new enhancement was addressing the need to create a system of feedback between stages. The second enhancement was the initial incorporation of *prototyping* in the software life cycle. The difficulties with the Waterfall model are well known today. In the words of Bill Curtis (A key person at the SEI—Software Engineering Institute) these difficulties are:

- It requires fully elaborated documentation
- It does not work well for the end-user application
- It reflects a manufacturing orientation
- It fails to treat software development as a problem-solving process.

The result of this realization was the development of more recent models, such as the *Incremental and Evolutionary models*. The *Spiral Model* was evolved from the Waterfall Model. (For details, refer "*Software Engineering: A Practitioner's Approach*" by Roger S. Pressman—McGraw-Hill International).

In summary, we say that it is important to note—the discipline of software engineering is still relatively new. Models and processes for software engineering are evolving and will continue to evolve. With this introduction, we now turn to the discussion on the history of "Maintenance Process".

C.2.2 History of the Maintenance Process

Maintenance projects continue to be a major constituent of software related businesses worldwide. In the context of the Indian Software Industry (Refer Appendix B), Maintenance is one of the most viable businesses next to only Skills Exports (the other often used term is "Body Shopping"). Thus, the maintenance work is important to us. This is the motivation to writing this section on maintenance.

"Operations and Maintenance" is usually the last phase in a Software/System Development cycle. This last phase occurs after product delivery. Often this is understood as a single simple step at the end of

the development cycle! Experts feel that "maintenance" would be more accurately portrayed as additional development stages. Basically, there is so much disagreement about what happens during maintenance and when development ends and maintenance begins! Because of this, the commonly held view is to think of maintenance as the murky, ill-remembered yucky area of the life cycle process. With this background, let us now discuss the Maintenance Models.

C.2.2.1 Early Maintenance Models

Since the 1970s, several software maintenance models have been proposed. Many of these are very similar. In general, each model has three stages:

- Understand the software
- Modify the software based on the specified requirements
- Re-validate (Test) the software

For *Corrective Maintenance*, the models focus on Problem Verification, Problem Diagnosis, Re-programming and Baseline Re-verification. Other software maintenance models propose several phases such as:

- Determine the maintenance objective
- Understand the program
- Generate a maintenance change
- Account for the ripple effect
- Conduct Regression Testing (The topic of "Regression Testing" has been discussed in Chapter 10)

C.2.2.2 Later Maintenance Models

With the increasing realization toward the need for standardization in software maintenance, the IEEE Computer Society Software Engineering Standards Subcommittee published the "IEEE Standard for Software Maintenance" (IEEE 1219). The standard detailed an iterative process for managing and executing software maintenance activities (www.ieee.com). The process model includes inputs, process, control and output for software maintenance. The standard states that maintenance planning should begin when planning for software development begins. However, the suggestion for planning and maintenance are just suggestions—they are not part of the standard. They are included in the standard's annex for information only.

If you were setting up a software maintenance organization and needed to implement a process model for maintenance, you would first ask, "Is there a standard or do we have to invent one?" A number of

Software Maintenance Standards do exist. The 20 most popular *software maintenance standards* are listed below:

Table 1

Maintenance Standards	Name
ISO/IEC 14764	Software Maintenance
ISO/IEC 12207	Information Technology-Software Life Cycle Processes
ISO/IEC 12207 — Amendment 1—	Amendment 1: Information Technology-Software Life Cycle Processes
ISO/IEC TR 15271	Guide for ISO/IEC 12207—(Software Life Cycle Processes)
ISO/IEC TR 15504-2	Guide for ISO/IEC 12207—(Software Life Cycle Processes)
IEC 60601-1-4	Medical Electrical Equipment—Part 1: General Requirements for Safety-4. Collateral Standard: Programmable Electrical Medical Systems
IEC 60880	Software for Computers in the Safety Systems of Nuclear Power Stations
IEEE 828	Software Configuration Management Plans
IEEE 1042	Software Configuration Management
IEEE 1219	Software Maintenance
IEEE 1540	Software Life Cycle Processes— Risk Management
AECL CE-1001-STD REV.1	Standard for Software Engineering of Safety Critical Software
AIAA ANSI/AIAA R-013	Recommended Practice for Software Reliability
BSI BS-5760-8	Reliability of Systems, Equipment & Components—Guide to Assessment of Reliability of Systems Containing Software
BSI BS-7738	Specification for Information Systems Products Using SSADM—(Structured Systems Analysis and Design Method)
DEF 00-55 Part 2	The Procurement of Safety Critical Software in Defense Equipment-Requirements
DEF 00-56 Part 2	Safety Management Requirements for Defense Systems Containing Programmable Electronics Part 2: General Application Guidance
German Process-Model (V-Model)	Software Life-Cycle Process Model (V-Model)
ASTM E 792	Guide for the Selection of a Clinical Laboratory Information Management System
ASTM E 1578	Standard Guide for Laboratory Information Management System
RTCA DO-178B/ED-12B	Software Considerations in Airborne Systems and Equipment Certification

C.2.2.3 The IEEE Maintenance Model

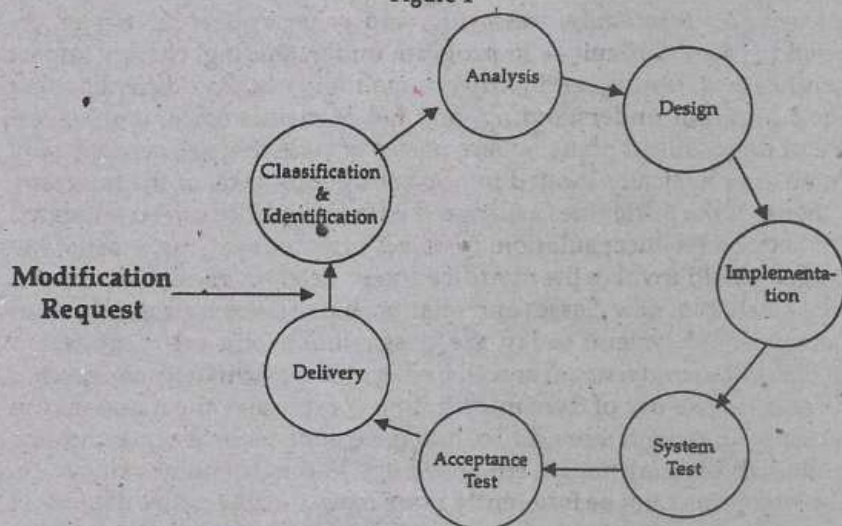
The IEEE defines software maintenance as the "process of modifying a software system or component after delivery to correct faults, improve performance or other attributes, or adapt to a changed environment." Software maintenance is the process that allows existing products to continue to fulfill their mission, to continue to be sold, deployed, and used, and to provide revenue for the development organization. Generally speaking, *maintenance* refers to all the activities that take place after an initial product is released. However, in common usage, people

apply the term *maintenance* not so much to major *evolutions* of a product, but regarding efforts to:

- Fix bugs (*Corrective Maintenance*)
- Add small improvements, or keep up with the state-of-the-art (*Perfective Maintenance*)
- Keep the software up-to-date with its environment: the operating system, hardware, major components such as DBMS, GUI systems, and communication systems (*Adaptive Maintenance*)

The figure below depicts the *IEEE Maintenance Process*.

Figure 1



The standard starts with the software maintenance effort during the post-delivery stage, as do all other maintenance models. It does not take into account the pre-delivery activities that are critical to cost-effective support of a software system. The complete discussion on each of the activities shown in figure above is beyond the scope of this book.

C.3 OO Maintainability

Most of the systems today are built using the object-oriented technology. One of the main arguments for using *object-oriented methods* is that it should reduce the amount of maintenance required on the software development (object orientation is known to promote "component reuse"). Several studies have been conducted to determine the impact of OO methods on maintainability. According to one study conducted in 1990, individual maintenance tasks are more difficult when using object-oriented software compared to the procedural software. On the other hand, there are others who support the premise that

object-oriented techniques result in fewer maintenance changes and reduced maintenance effort overall (Object Oriented Effort Points or OOEP are often used in measurements related to object-oriented systems).

However, there is not much empirical data upon which to base claims about the effectiveness of OO as far as the maintenance effort is concerned. According to some experts, *inheritance* makes dependencies among objects harder to find and analyze. Additionally, the greater the *depth of inheritance*, the greater the level of difficulty associated with understanding and maintaining the software. Thus, "Inheritance" is one of the key factors in determining maintainability. Greater levels of inheritance help *re-use*, but decrease maintainability.

Features of the object oriented paradigm such as *dynamic binding*, *encapsulation*, *inheritance*, *messaging*, and *polymorphism* have been reported to cause difficulties in program understanding, change impact identification, testing, and software maintenance. One difficulty that affects program understanding, and hence maintenance, is the presence of *de-localized plans*: where pieces of code that are conceptually related are physically located in non contiguous parts of the program.

Some of the difficulties associated with inheritance can be mitigated with better data encapsulation. In object-oriented systems, most of the changes would involve the modification or deletion of existing classes, or the creation of new classes and relationships between classes. In most object-oriented systems today, the class relationships are complicated and difficult to understand, and changing a class might have unexpected side effects. The use of dynamic binding complicates the maintenance process by making it more difficult to trace dependencies. In summary, maintainability may not get enhanced for Object-Oriented systems; i.e. OO systems may not be inherently more maintainable. More data must come forth, and more research is needed in this area (effect of object-orientation on system maintainability).

C.4 Maintenance in e-Business Era

C.4.1 e-Business Models

In today's digital era, most businesses are done on the Web with the e-Commerce paradigm. e-Business models play a vital role. Portals and sites are the gateway to the companies and their businesses. Since they play such a crucial role in generating revenue through the e-Business, these sites/portals have to be well maintained—uptime must be very high (almost 100%). Most of the time, the activity of Web-site maintenance is outsourced mostly to off-shore partners (This counts for a big business chunk for India). The entire discussion in this section is based on this fact.

Although a full discussion on various kinds of e-Business models does not form the scope of this book, we have listed below a number of

e-Business models proposed. The basic categories of business models mentioned in the table below include:

- Brokerage
- Advertising
- Infomediary
- Merchant
- Manufacturer

These e-Business models are implemented in a variety of ways. Moreover, a firm may combine several different models as part of its overall Internet business strategy.

C.4.2 Web site's Life Cycle

Given the importance of the sites and portals in the e-Business era, managing the Web Sites becomes crucial. Web-site maintenance is a big time activity in the e-Business era.

At any point in time your Web site is either being planned, under development or re-development, being launched, maintained, evaluated, enhanced and promoted or being assessed with a view to undergoing a re-birth. These stages form a "life cycle" which is rather like that of a garden's life cycle: the initial planning; each section planted out according to a schedule; plantings monitored and tendered to; sections developed further to cater for growth and the emergence of new trends in gardening. Any gardener knows that this life cycle requires constant management if it is to yield rewards. This is true of web sites too.

At any point a Web site is located somewhere in the Life Cycle as illustrated below. The eight phases identified in the *Web Site Life Cycle* are independent. Acknowledgement of the Web Site Life Cycle and the need to manage it dispels the myth that developing and managing a Web site finishes on the day it is launched—the launch is simply the end of the beginning. Successful Web sites are in a state of perpetual change, which requires systematic management. The Web Site Life Cycle is depicted in the figure on the next page.

PLANNING:

- Savings & Revenues (e-Commerce, Cost Savings and Revenues)
- Promotion (search engines, the launch, advertising, portals, repeat visitors)

BUILDING:

- Engaging External Web Developers (Project Plan, Request for Proposal, Development Contract, Developer selection)

- Managing the development (Staging the Development, Managing the deliverables, Testing Procedures)

MAINTENANCE (Technical maintenance, content updating, evaluating and enhancing, resources, tool and budgeting)

MANAGEMENT (the Web Site Life Cycle, the management team, managing people, resources, budgets & risks)

END USER PERSPPECTIVE

- The Users' experience (aims, audiences, naming, content, functionality, technical issues)
- The Design (accessibility, information and graphics design, other technical issues)



C.4.3 Why Manage the Life Cycle?

The elements that require managing will usually fall into three main categories:

- **People**—management, curators and artistic directors, marketing, education officers, IT, clerical, Web designers and authors (both in-house and external)
- **Resources**—time, hardware and office equipment, databases, space
- **Finances**—money for in-house staff salaries, fees to external designers, programmers etc, software licenses, copyright clearance

Managing all the elements of each category effectively will help to ensure that:

- What you try to achieve in each stage of the Life Cycle is realistic and achievable given the available people, resources and finances
- Each stage of the Life Cycle is completed on time and within budget
- People in your organization are not over-stretched for time nor expected to undertake tasks for which they are not qualified or skilled—all of which leads to stress and unhappy people
- Each stage of the Life Cycle is carried out in the most efficient manner leading to maximum saving of money and resources, e.g. resources and money are not wasted due to poor timing or sequencing of events
- Each stage of the Life Cycle is actually undertaken, e.g. evaluation conducted regularly
- Risks are foreseen and managed appropriately, e.g. hold-ups due to difficulty in securing reproduction rights.

C.4.4 Who Should Manage the Web-Sites

The Management Team—The team's role is to consider all matters related to the *Web Site Life Cycle*. The Management Team consists of people representing at least the following areas within the organization:

- Management, e.g. director, head curator, artistic director
- Curatorial staff /practitioners
- Marketing/promotions
- Education
- Information services/technology

The Online Manager—The Online Manager is the "head gardener" of the *Web Site Life Cycle* (i.e. his efforts decide how the site will "bloom"!).

Issues to consider when choosing the Online Manager:

- Personal attributes
- Role and responsibilities
- Skills and knowledge

Managing the People—The *Web site Life Cycle*, from the planning stage through the building phase to the evaluation and maintenance stages, is demanding on your organization's staff. It requires:

- **Time**—meetings, responding to emails, design and test prototypes etc
- **Energy**—to contemplate the issues, engender enthusiasm

- **Vision**—what new ways of presenting exhibitions might be possible
- **Research**—marketing, audience needs, technology adaptation
- **Analysis**—technology platforms, usability testing
- **Coordination**—of others in the organization to gain their opinions and input; content scheduling; redirecting existing work to produce content for the site
- **Understanding**—of the internet and the expectations of the users
- **Patience**—everything takes longer than you think it will!
- **Tolerance**—of other points of view and requirements of other departments in your organization

If insufficient consideration is given to the level and kinds of input required of staff throughout the Web site Life Cycle—i.e. not merely through the building of the site—then its effectiveness will suffer, as will the people involved.

Roles in the Web-Site Life Cycle—Who is needed

These are the roles that have to be filled if the Life Cycle is to be maintained effectively:

- Project director
- Content editor
- Webmaster/technician
- Research officer
- Content technician
- Librarian
- Marketing/promotions manager
- Trainer
- Financial controller
- Legal advisor
- Graphic designer
- Web programmer

Some of these roles require full-time positions while others may only be required during certain stages of the Life Cycle. Also some of above roles may get combined (for example the Webmaster and Web Programmer, the Content Technician and the Graphic Designer). For many smaller organizations one or two people may have to undertake most of these roles.

To Outsource or not to Outsource?

How much of the Web development and management can or should be conducted in-house and how much should be outsourced? Issues to consider are:

- Keep to your core business or learn new skills?
- Skills transfer
- Training required and new job descriptions
- Time allocations

Maintaining the Right Attitude: Building a Web Culture

Possible strategies to encourage a Web culture in your organization:

- Workflow engineering
- Awareness raising
- Creating ownership
- What competitors are doing

Managing the Resources—What resource management issues will you need to address? Consider these:

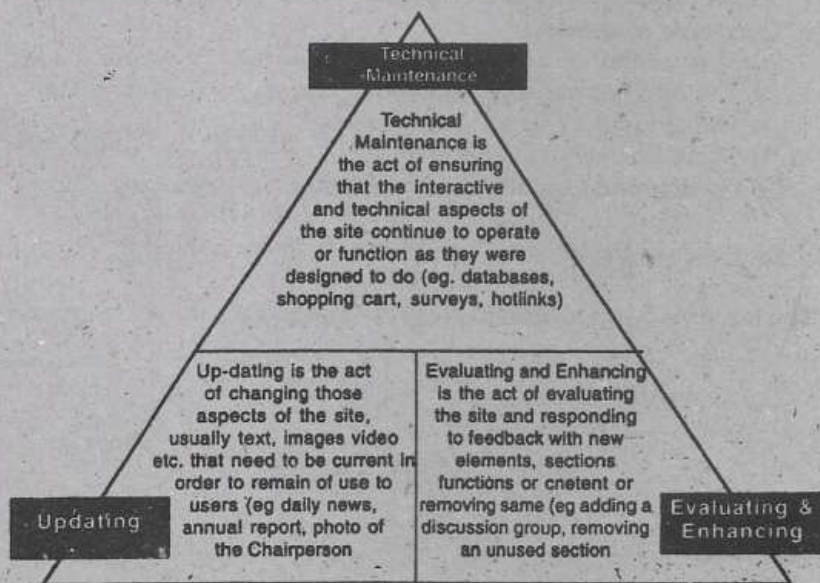
- Communications, hardware and software requirements
- Time
- Office space and equipment

C.4.5 Managing the Maintenance of Your e-Business Site

C.4.5.1 The Site Maintenance Triangle

The following Site Maintenance Triangle recognizes that web-site maintenance consists of three principal elements: **Technical Maintenance**,

Figure 2



Updating, Evaluation & Enhancing. Considering site maintenance in the light of these categories encourages appropriate planning and resource allocation thereby minimizing the risk of overlooking or underestimating maintenance issues, costs, resources required and responsibilities.

Technical Maintenance is the act of ensuring that the interactive and technical aspects of the site continue to operate or function as they were designed to do.

C.4.5.2 Managing the Maintenance of Your Site

The Management Team established early in the life cycle should meet at least monthly after the launch of the site to:

- Review/suggest new content (where appropriate) for inclusion in the site
- Identify where Web site enhancement/development is required
- Review relevant activities of the marketing and promotions group
- Assess the statistics and feedback on the site from all sources (e.g. contact-us, Web server, questionnaires, anecdotal) and identify necessary action to remedy problems and/or highlight strengths
- Review Internet policies (including the legal aspects see [16] at the end of this appendix) and procedures and develop new ones where required
- Evaluate the success of the site against its stated aims and objectives, e.g. to improve public awareness of the organization's products and services, reduce publishing costs etc—criteria may include such things as:
 - o Feedback from users via the site and the usual formal and non-formal channels
 - o The volume of traffic through the site month by month
 - o The number of publications, fact-sheets etc downloaded
 - o The amount of revenue generated
 - o Savings in other areas e.g. publishing budget
 - o Quality and quantity of contributions from stakeholders

C.4.5.3 Day-to-Day Maintenance of an e-Business Site

The table below identifies the responsibilities of the Management Team

Table 2

Function	Twelve Month Maintenance Tasks	Estimated Working days per month	Estimated Total Days per year
Project Director	Chairs Management Committee: policy decisions and resource allocation, budgets, legal and contractual issues, overall responsibility for the site		
Manager	Responsible for all day-to-day operations of the site, oversees the Webmaster, coordinates contributions of content, implements policies and guidelines, monitors usage of the site, initiates and oversees further development		
Content Editor	Ensures integrity of content, editing, creating, collating of content, uploading new content to the site		
Site Marketer	Implements marketing strategy, develops new strategies as appropriate		
Content Contributors	Provides content to the Content Editor from their particular area of the organization, e.g. new images, exhibition		
Writer	Creates new text as requested		
Webmaster	Maintains technical functions of the site and provides assistance to Content Editor as required, maintains the database mailing lists etc, reports on hits and traffic		
Outsourced Web Developer	Incorporates new features and designs new graphical elements as directed		
Outsourced Project Advisors	Provides ad hoc online strategic advice, legal and marketing advice		

C.4.5.4 Perfective Maintenance for e-Business Web Sites

Identifying What Requires Updating on the e-Business Sites/Portals

The table below is not exhaustive in its listing of content, graphics etc. It is intended to be used as a template for Web site managers responsible for the maintenance to draw up their own list of what needs up-dating and maintaining and how frequently. It will be helpful to use the site map as a guide to ensure that all areas of the site are covered in the table:

Table 3

<i>Maintenance Frequency</i>	<i>Content</i>	<i>Graphics</i>	<i>Functionality Examples</i>	<i>Technical Implications</i>
Daily	Daily news items	Images from performances, galleries	Discussion groups	Monitor hits and traffic
			Contact us	Export data collected from forms to the
			Shopping cart Bookings for seats, events etc	appropriate personnel Check error reports from web server
Weekly or Monthly	Bulletins	Images from performances, galleries	Links	Email list servers
	Articles	Promotional banners	Search facility	E-commerce system Database configuration
	e-newsletters			Report on hits and traffic
As required	Competition Tracking			Re-register with search engines
	Introductory text	Photographs	Post-cards	Report on data collected from surveys
	About us	Illustrations	Maps	Maintain Web server
	Contact details	Diagrams	Navigation elements	Play-back feature for audio/video
	Educative material	Logos	Password function	Auto-email back
	Audio		Animations	Ensure integrity of file naming conventions
	Video		Uploading PDFs (Portable Document Formats)	
	Prices Items for sale			Maintain metadata
Annually	Annual Reports Annual events: e.g. festivals (Seasonality of Business)	Review compatibility with corporate image	Review possible new functionality	Review impact of new tools & solutions

C.4.5.5 Quality Issues in Web-Site Maintenance

Quality Control of Updating—The updating of content should not be conducted without some checks and balances in place. The Content Editor should be assigned the responsibility for verifying the quality and accuracy of new content for the site as supplied by others and for

ensuring it is up-loaded to the right place and in a timely manner. Some tips for the quality assurance procedure:

- Establish a pro-forma that all contributors to the Web site use to submit new content and/or changes to existing content and/or functions, specifying such things as:
 - The format for the content
 - How to indicate requested links
 - Where in the site the new or revised content is to be placed
- Confirm that the content supplied for up-dating does not infringe anyone's copyright, IP etc
- Proofread the new content and seek approval from the author for changes prior to up-loading
- Determine whether new content should be signed-off by the Management Team
- Use the updating templates created by the Web developers and/or any necessary software as purchased for the purpose to facilitate updating

Updating Tools

There are numerous tools available on the Web free or at a very reasonable cost that will make maintenance and updating relatively easy. User-friendly HTML editors produced by some of the biggest names in the Internet that can be purchased and downloaded include:

- Macromedia Dreamweaver—<http://www.macromedia.com/software/dreamweaver/>
- Microsoft FrontPage—<http://microsoft.com/frontpage/>
- Adobe GoLive—<http://www.adobe.com/products/golive/main.html>

The Web developer that built the site, or other of standard competence, should be able to build various templates that can be accessed from the desktop and enable text, images, audio or video files to be inserted into the template for uploading to the site.

C.4.5.6 Evaluating and Enhancing Your Site

How do you know in what ways the site could be enhanced? Is it anecdotal evidence, personal assessment or feedback from your users or a third party engaged to evaluate your site? Each of these is a valid means of identifying how the site could be improved. The evaluation process needs to be systematic and managed as an integral part of the Web Site life cycle. Techniques that can be employed in this regard include:

- Feedback from the site
- Questionnaires
- Focus-groups
- Monitoring site activity

Monitoring can inform you as to what is working, what is popular and where the "dead" areas are that require rejuvenation or cutting. The following information can be gained from monitoring software:

- How many people access the site
- The pattern of access over 24 hours, a week etc.
- What country are the users from
- What areas of the site are visited the most and the least
- What routes users take through the site
- How long they stay in the site
- What point they exit from the site
- The existence of broken links

The following sites offer tools that can be downloaded or Web sites that offer a monitoring and reporting service:

- WebTrends—www.netiq.com/webtrends/
- Maxamine—www.maxamine.com
- Microsoft Site Server 3—<http://www.microsoft.com/siteserver/site/default.htm>
- My Mind-it—http://www.pumatech.com/press_releases/012202.html

C.4.5.7 Technical Maintenance

Recall the figure "Site Maintenance Triangle" in section C.4.5.1. Technical Maintenance is the act of ensuring that the interactive and technical aspects of the site continue to operate or function as intended—i.e. maintaining the technical status quo. This might include:

- Responding to faults with the database, search capacity on the site, shopping cart feature, discussion groups etc
- Ensuring online forms are working and that data from them is being passed to the correct person
- Monitoring the security of the site
- Registering and re-registering the site with search engines
- Responding to emails from users regarding technical faults on the site
- Ensuring the site works for new versions of Internet Explorer and Netscape
- Checking hotlinks to external sites to ensure they are still valid

The person assigned to the technical maintenance role requires a sound level of technical expertise and experience with the World Wide Web. This person is usually termed the "webmaster" and is an integral part of the Management Team.

C.5 Cyclomatic Complexity

C.5.1 What is Complexity?

Complexity

1. *Apparent Complexity* is the degree to which a system or component has a design or implementation that is difficult to understand and verify
2. *Inherent Complexity* is the degree of complication of a system or system component, determined by such factors as the number and intricacy of interfaces, the number and intricacy of conditional branches, the degree of nesting, and the types of data structures.

C.5.2 What is "Cyclomatic Complexity"?

Cyclomatic complexity is a *measure of software complexity* first introduced by Thomas McCabe in 1976 and measures the number of *linearly independent paths* through a program module. A program module is generally considered to be a procedure having a single entry and exit point. Each program instruction that can potentially cause the execution path to change, such as an "If... Then" statement, increments the complexity measure.

C.5.3 How is it Different than Other Software Measures?

Software complexity is one branch of *software metrics* (The topic of Software Measurements and Metrics was discussed in Chapter 5) that is focused on direct measurement of software attributes, as opposed to indirect software measures such as project milestone status and reported system failures. There are hundreds of *software complexity measures*, ranging from the simple, such as source lines of code, to the esoteric, such as the number of variable definition/usage associations.

C.5.4 Cyclomatic Complexity: Implications for Maintenance and Testing

"Complexity" has some relation to software maintenance and to testing. For instance, research has shown that minimizing complexity is a key to writing high quality code. If the *cyclomatic complexity* of software is measured regularly during construction then complex areas of

code can be identified early in the development life cycle and re-structured to improve understanding and to reduce testing and future maintenance effort.

The cyclomatic complexity measurement of a procedure equates to the number of tests required to execute all code paths through a procedure and is therefore of use to developers whilst unit testing. It is commonsense following that higher the complexity, the more difficult becomes the software to test. The cyclomatic complexity of existing software can also be measured to assess possible risk during modification and the level of testing required.

C.5.4.1 Cyclomatic Complexity as related to Software Maintenance

Maintenance is the most expensive phase of the software lifecycle, with typical estimates ranging from 60% to 80% of total cost. Consequently, maintenance methodology has a major impact on software cost. "Bad fixes," in which errors are introduced while fixing reported problems, are a significant source of error. Complexity analysis can guide maintenance activity to preserve (or improve) system quality, and specialized testing techniques help guard against the introduction of errors while avoiding redundant testing.

Complexity tends to increase during maintenance, for the simple reason that both error correction and functional enhancement are much more frequently accomplished by adding code than by deleting it. Not only does overall system complexity increase, but the complexity of individual modules increases as well, because it is usually easier to "patch" the logic in an existing module rather than introducing a new module into the system design.

Cyclomatic complexity usually increases gradually during maintenance, since the increase in complexity is proportional to the complexity of the new code. For example, adding four decisions to a module increases its complexity by exactly four. Thus, although complexity can become excessive if not controlled, the effects of any particular modification on complexity are predictable.

C.5.4.2 Complexity as related to Software Testing Effort

There is a direct relationship between the cyclomatic complexity value and the number of test cases that need to be devised and executed to fully test a procedure. Cyclomatic complexity values can therefore be used to identify which areas of the software will require greater testing effort than others. The lower the cyclomatic complexity, the easier it is to determine suitable tests and to perform them.

As cyclomatic complexity measures the amount of decision logic in a single software module, it is used for two related purposes in the

structured testing methodology. First, it gives the number of recommended tests for software. Second, it is used during all phases of the software lifecycle, beginning with design, to keep software reliable, testable, and manageable.

C.5.5 Acceptable Level of Complexity

Many organizations have found it useful to set a threshold complexity value that all developers must stay within—a value less than 10 is widely regarded as an acceptable value, although research has shown that for highly skilled developers, expressing commitment to increased testing, values less than 15 may occasionally be acceptable). Studies show that the lower the cyclomatic complexity for a procedure, the easier it is for the evaluator to understand the intended purpose of the procedure.

C.5.6 Cyclomatic Complexity and Modification Risk Assessment

The cyclomatic complexity of a procedure can also be used as an indicator of the possible risk associated with modifying a procedure. Higher complexity will normally result in higher cost and risk when making modifications. The following table assesses the risk of modification to a procedure:

Table 4

Cyclomatic Complexity	Risk Assessment
1-10	Simple procedure, low risk.
11-20	More complex procedure, moderate risk.
21-50	Complex procedure, high risk.
Greater than 50	Un-testable procedure, very high risk

Cyclomatic complexity is based entirely on the structure of software's control flow graph.

C.5.7 Control Flow Graphs

Control flow graphs describe the logic structure of software modules. A module corresponds to a single function or subroutine in typical languages, has a single entry and exit point, and is able to be used as a design component via a call/return mechanism. Each flow graph consists of nodes and edges. The nodes represent computational statements or expressions, and the edges represent transfer of control between nodes.

Each possible execution path of a software module has a corresponding path from the entry to the exit node of the module's control flow

graph. This correspondence is the foundation for the structured testing methodology.

Cyclomatic complexity is defined for each module to be $e - n + 2$, where e and n are the number of edges and nodes in the control flow graph, respectively. For more details on the topic of "Cyclomatic Complexity" readers may like to refer any standard textbook on Software Engineering.

C.6 Requirements

C.6.1 What's the big deal about 'requirements'?

One of the most reliable methods of insuring problems, or failure, in a complex software project is to have poorly documented requirements specifications. Requirements are the details describing an application's externally perceived functionality and properties. Requirements should be clear, complete, reasonably detailed, cohesive, attainable, and testable (A good standard for writing requirements specification is the *IEEE Guide to Software Requirements, Specification Std 830*). A non-testable requirement would be, for example, 'user-friendly' (too subjective). Ambiguous requirements cause problems for validation i.e. testing. A testable requirement would be something like 'the user must enter their previously-assigned password to access the application'. Determining and organizing requirements details in a useful and efficient way can be a difficult effort; different methods are available depending on the particular project. Many books are available that describe various approaches to this task (one such is the [5] at the end of this appendix.

C.6.2 Requirements Taxonomy

A requirement is a specification of what a system must do—the things/behaviour of the systems users can observe. Sometimes, this is called the *architecture* of the system. Good requirements have three primary characteristics (the IEEE Std 830 mentioned above has the details on this):

1. They are *precise*, with no possibility for misinterpretation by users or implementers
2. They specify *what* the system must do, not *how* to do it. They avoid specifying implementation details
3. They show *conceptual integrity*, building on a simple set of facilities that interact well with each other.

The figure below presents a matrix for "requirements taxonomy":

Figure 3

	Customer Doesn't Care	Customer Cares	
		Measurable	Un-measurable
Observable To Users	Requirement likely to change	Requirement	Goal
Not Observable To Users	Implementation detail	Constraint	

While defining the systems requirements, an architect may also define "constraints" (the other word for this is, "nonfunctional requirements") and goals for the system. A constraint is a limitation on possible implementation of the system (For example, a particular implementation language or a particular algorithm insisted upon). From project management point of view, it is often better to negotiate away constraints, since they limit the freedom of the implementer.

A goal is a statement that guides tradeoffs among design decisions. For example, a customer may care a lot about the maintainability of the software, but may not care so much for efficiency. A point to note is that a goal may become a requirement if you can find a way to "quantify" it. For example, "high throughput" is a goal, but "at least 50 transactions per second" is a requirement. One lesson we draw from this is that if something is not quantifiable, it may become difficult to test causing problems in the later phases of the project!

Thus, care should be taken to involve ALL of a project's significant stakeholder (mainly 'customers') in the requirements process. 'Customers' could be in-house personnel or outsiders, and could include end-users, customer acceptance testers, customer contract officers, customer management, future software maintenance engineers, salespeople, etc. Anyone who could later derail the project if their expectations aren't met should be included if possible. From this perspective the "requirements taxonomy" presented above is useful.

Organizations vary considerably in their handling of requirements specifications. Ideally, the requirements are spelled out in a document with statements such as 'The product shall.....'. 'Design' specifications should not be confused with 'requirements'; design specifications should be traceable back to the requirements. This is an important angle from the Integrated CMM perspective which expects something known as "bi-directional traceability" (The Process Areas called *Requirements Development* and *Requirements Management* of the CMMI—see Chapter 9) Readers may also like to note that the IEEE Standard 1471:2000 is about "Architecture Design for Software Intensive Systems"

In some organizations requirements may end up in high-level project plans, functional specification documents, in design documents, or

in other documents at various levels of detail. No matter what they are called, some type of documentation with detailed requirements will be needed by testers in order to properly plan and execute tests. Without such documentation, there will be no clear-cut way to determine if a software application is performing correctly!

Finally at the end of this section, we present the various aspects or activities involved in "Software Requirements":

Software Requirements

- Requirements *Engineering* Process
 - Process Models
 - Processes and Actors
 - Process Support and Management
 - Process Quality and Process Improvement
- Requirements *Elicitation*
 - Sources of Requirements
 - Elicitation Techniques
- Requirements *Analysis*
 - Requirements Classification
 - Conceptual modeling
 - Architectural Design and Requirements Allocation
- Requirements *Specification*
 - Requirements Definition Document
 - Software Requirements Specification
 - Document Structure and Standards
 - Document quality
- Requirements *Validation*
 - Requirements Reviews
 - Prototyping
 - Model Validation
 - Acceptance Tests
- Requirements *Management*
 - Change Management
 - Requirements Attributes
 - Requirements Tracing

The detailed discussion is beyond the scope of this book. Some good references on the topic of "Requirements" are indicated at the end of this appendix.

C.7 Principle of COUPLING and COHESION

C.7.1 Background and Introduction

Larry Constantine and Edward Yourdon discussed *Coupling and Cohesion* at great length in their 1976 landmark book, *Structured Design*.

"Coupling and Cohesion" are fundamental design principles. Since these concepts have been tied to structured programming, in today's object-oriented scenario, it seems that many programmers are forgetting what they mean. In many ways, the concepts are still extremely important as fundamental design concepts. The aim of the designer or system architect is supposed to be "High COHESION and Low "COULPING".

The design of software is often depicted by graphs that show "components and their relationships". For example, a structure chart shows the calling relationships among components. *Object-oriented design* too is not an exception to this. Coupling and Cohesion are attributes that summarize the "degree of interdependence or connectivity among subsystems/modules and within subsystems/modules", respectively.

When used in conjunction with measures of other attributes, coupling and cohesion can contribute to an assessment or prediction of software quality. These measures have the properties of system-level coupling and cohesion defined by Briand, Morasca, and Basili.

Coupling is based on relationships between modules. We can have a measure for intra-module coupling based on an intra-module abstraction of the software, rather than inter-module. Intra-module coupling is calculated in the same way as inter-module coupling. We can define cohesion in terms of intra-module coupling, normalized to between zero and one.

C.7.2 Coupling and Cohesion in Measurement Context

Inter-module Coupling

It is the measure of the interdependence of one module to another. Modules should have low coupling. Low coupling minimizes the "ripple effect" where changes in one module cause errors in other modules. This helps minimize issues during Regression Testing (The topic of Testing is covered in great detail in Chapter 10).

Inter-module Cohesion

It is the measure of strength of the association of elements within a module. Modules whose elements are strongly and genuinely related to each other are desired. A module should be highly cohesive.

C.7.3 Cohesion

We will discuss cohesion first, since it is a bit easier to understand. As said earlier, "Cohesion" refers to how clearly defined a particular module or procedure is. A module with high cohesion does one or a few

things exceedingly well. Let's take an example of a job description. Suppose you placed the following ad:

Wanted: Programmer with solid skills in the following areas: System analysis, design, testing methodologies, EDI, software auditing, COBOL, BASIC, PAL, SmallTalk and C++ required. Strong real-time data acquisition skills, scientific applications experience and accounting knowledge a must. Candidates must be experienced, energetic, willing to work long hours, be able to lead groups of programmers, get along with others and have the patience of Job.....

If this is the way you stated the requirement, how many qualified candidates would you get? And if you found one, how long would he or she last? How many skills can one person excel in at one time?

What this means is that if a particular module is responsible for a diverse set of tasks, chances are the module will be, like our hapless candidate in the above example, a mess!

So the question that comes up is "what constitutes a module, which exhibits poor cohesion?" From this, there are a few points to note:

Concept of Cohesion goes a lot with the principle of "divide and rule" and "work-breakdown structure. As you can see, the first sign, usually, that a module exhibits poor cohesion is its "length". The longer a programmer rambles on in a module, the drearier and less comprehensible the module becomes (This also adversely affects code maintainability). The module lacks the punch, (which of course, is a good thing for keeping those who maintain the code awake and get paid for doing this!).

As an illustration to drive this point, if a program module is coded to perform too many different things: getting input from the user (on whether to continue or not, messaging the user on error conditions, performing several queries and setting up the query images and locking tables. Each of these tasks can be delegated down in the hierarchy to a specialized module. In the Object-Oriented paradigm this would mean writing short but focused methods to handle only a particular type of task. Such an approach would improve the module's cohesiveness.

In life, there are exceptions to the rule. This is true here too. It is possible for a lengthy module to still have a high degree of cohesion, *provided the module excels in one or two things only*. For example, some modules contain code, which executes a series of commands or a series of evaluations. The latter case is especially common in situations where the module needs to evaluate a list of menu choices or a list of events. For example, a long series of case statements evaluate the user's menu choice. Since this kind of a module would exactly one thing, *evaluate a menu choice*; it qualifies as a highly cohesive module.

Thus, as a good design principle, it is a good idea to decompose the programming problem into a hierarchy of cooperating procedures.

Clearly identify the job description for each module. Try to keep each module as painfully ignorant as possible of what the other modules are doing (In Object Oriented paradigm, this is the concept of "hiding").

After all, a number of programmers would be assigned various modules to code. How many times have you looked at your own code, written months ago, baffled by what it does? If you clearly identify a narrow job description for each procedure, the chances for confusion later are less.

It might be helpful to keep in mind the following "Rule of Module Intelligence":

Bosses are ignorant. Workers are smart

(Readers from the older SSADM—Structured System Analysis and Design days would recall the "Functional Decomposition Diagram or the FDD).

Just like in real life, endowing your various modules with different skill helps keep cohesion high. For example, the higher you go in a module hierarchy, the less nitty-gritty work the module actually does. In fact, higher-level modules usually bark out commands, as the module in the listing below (hypothetical):

Code Listing—High-Level Module

```
procedure BIG_BOSS()
  if UNIVERSE_CREATE_THYSELF() = false then
    return false
  endif
  PROCESS_MAIN_MENU_SELECTION()
  if UNIVERSE_GO_AWAY() = false then
    SHOW_MESSAGE("Something bad happened during universe
      shut-down")
    return false
  endif
  return true
endproc
```

In this example, BIG_BOSS() initializes the application, calls the procedure which displays the main menu and handles the user's selection and then shuts the application down. This module does very little except issue a few key commands to subordinates.

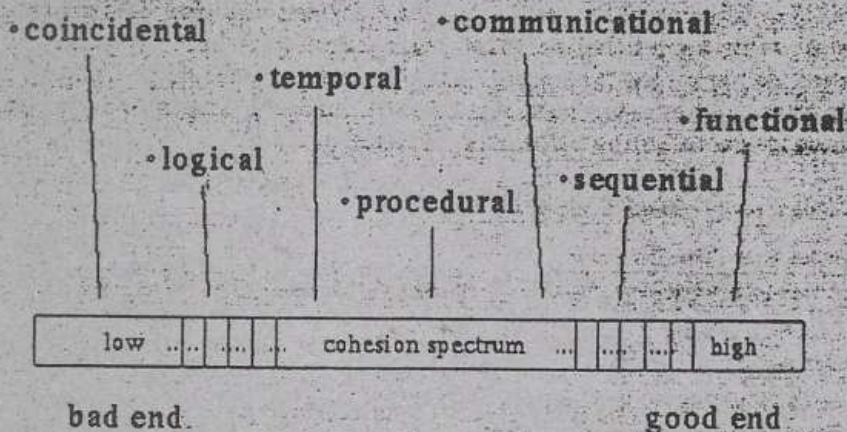
Main procedures like BIG_BOSS() are fairly easy to write without losing cohesion. The real challenge is writing in lower-level procedures properly. Often, programmers use the procedure they are working on as a sort of scratch pad to explore possible alternatives to a solution.

And often, a less experienced programmer will try to patch-up the scratch pad or cobble together the random bits and pieces of code into a working single module.

Having discussed the basic concept behind "Cohesion", let us now consider various "types of Cohesion"

C.7.3.1 Types of COHESION

As discussed above, "Cohesion" is the measure of strength of the association of elements within a module. Modules whose elements are strongly and genuinely related to each other are desired. Thus, a module should be highly cohesive. There is a "Cohesion spectrum" as illustrated below:



[1] Coincidental cohesion

A module has *coincidental cohesion* if its elements have no meaningful relationship to one another.

Example:

A perhaps mythological story—A manager who is responsible for maintaining a largish (say, 10000 line of code) FORTRAN program reads about modularity and how helpful it is in improving programming understanding of programs.

The manager tells his chief programmer to restructure the FORTRAN program (which is a one large subprogram) into modules. He's going to get those benefits.

The chief programmer, who thinks he has better uses for his time, decides the fastest way to make modules out of the program is to take

a ruler, measure down every 6 inches in the program listing and draw a line. At each line, he inserts a call to a new subprogram, and after that call creates the header for the new subprogram. With this short cut approach taken, these "modules" exhibit ultimate coincidental cohesion!

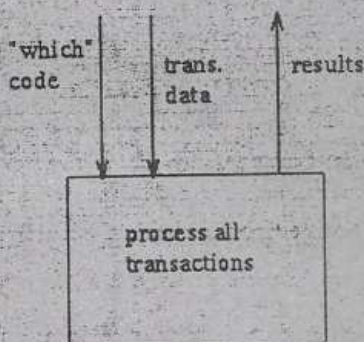
[2] Logical cohesion

A *logically cohesive* module is one whose elements perform similar activities and in which the activities to be executed are chosen from outside the module. See the diagram below.

logical

A logically cohesive module is one whose elements perform similar activities and in which the activities to be executed are chosen from outside the module.

Example:



The actions in the module are all logically "transaction processing". The "which" flag tells the module what part of its internal logic to apply to the particular transaction data coming in for each specific invocation.

[3] Temporal cohesion

A temporally cohesive module is one whose elements are functions that are related in time. Example:

Consider a module called "On_Really_Bad_Failure" that is invoked when a "Really_Bad_Failure" happens. The module performs several tasks that are not functionally similar or logically related, but all tasks need to happen at the moment when the failure occurs. The module might

- Cancel all outstanding requests for services
- Cut power to all assembly line machines
- Notify the operator console of the failure
- Make an entry in a database of failure records

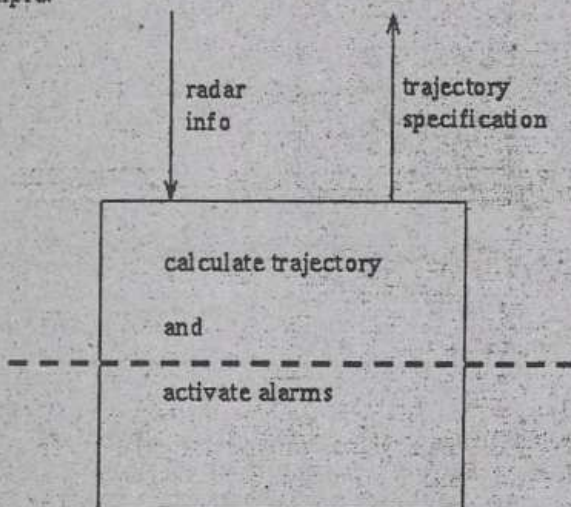
[4] Procedural cohesion

A *procedurally cohesive* module is one whose elements are involved in different activities, but the activities are sequential. See the diagram below.

procedural

A procedurally cohesive module is one whose elements are involved in different activities, but the activities are sequential.

Example:



Why might we want these as two separate modules (say, cut along the dotted line)?

Then we could calculate trajectories for other purposes without risking alarms going off... or, similarly, we could set off alarms without having to have some target to track first.

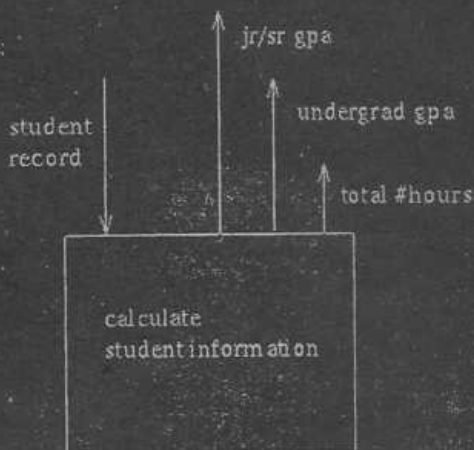
[5] *Communicational cohesion*

A *communicationally cohesive* module is one whose elements perform different functions, but each function references the same input information or output. See the diagram below.

communicational

A *communicationally cohesive* module is one whose elements perform different functions, but each function references the same input information or output.

Example:

[6] *Sequential cohesion*

A *sequentially cohesive* module is one whose functions are related such that output data from one function serves as input data to the next function. See the diagram on the next page.

[7] *Functional cohesion*

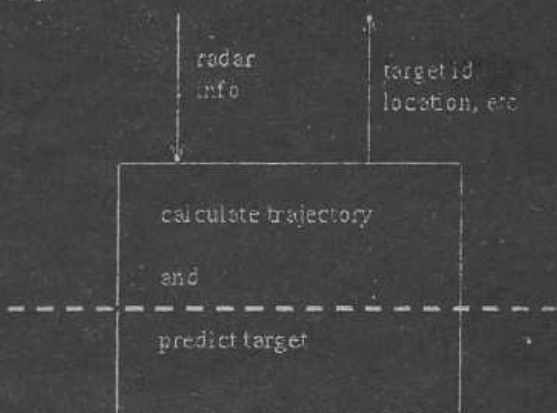
A *functionally cohesive* module is one in which all of the elements contribute to a single, well-defined task. Object-oriented languages tend to support this level of cohesion better than earlier languages do. Examples of functionally cohesive modules are:

- Calculate Net Employee Salary
- Read a Transaction Record
- Assign seat to a passenger
- Compute point of collision of two objects
- Verify syntax

sequential

A sequentially cohesive module is one whose functions are related such that output data from one function serves as input data to the next function.

Example:



Is this case, would there be value in "cutting" along the dotted line? Not necessarily... can you envision calling "predict target" without having immediately prior to that called "calculate trajectory" to acquire the information needed by "predict"? The second task needs the first.

C.7.4 Coupling

The other design issue, "Coupling", refers to what information needs to be communicated between modules. In real life jobs, most bosses prefer to have workers that can perform their duties without excessive communication, either spurious or required. The more *self-reliant* the worker, the less maintenance the worker will require.

Procedures need to communicate data between themselves. In fact, communicating data between modules is necessary to write anything other than a trivial program, such as the famous "Hello world" program shown in most textbooks on programming. Procedures transform data for the user. To do that, they must share data and COUPLING is about data sharing through parameters or messages exchanged between two or modules/classes.

Writing modules with low coupling is desirable. Why—Because a module with low coupling is more independent and usually less complex because there is less data for the module to work on. The less a module has to know about data in other modules, or in any other part of the application, the better.

Coupling, Cohesion, Span and life of a variable can be discussed in simple terms. The problem can be stated this way:

1. Smaller procedures are easier to understand because they contain less visual and hence logical information. This creates less complications..
2. The more information about a program that is immediately available to the programmer's eye at one time, the easier the program is to understand and review.
3. The more order and structure inherent in the information about a program, the easier it is to understand. The more random or chaotic the information in a program, the more difficult it is to understand, review or modify.

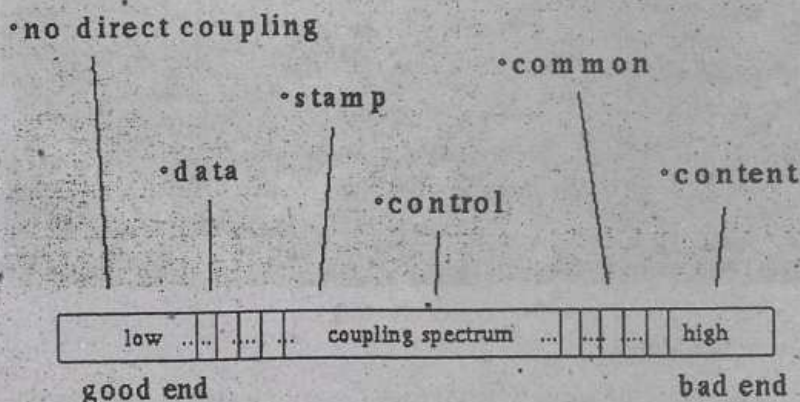
Having discussed the basic principle behind "Coupling" let us now consider various "types of Coupling".

C.7.4.1 Types of COUPLING

As discussed earlier, *Coupling is the measure of the interdependence of one module to another.* Modules should have low coupling. Low coupling minimizes the "ripple effect"

where changes in one module cause errors in other modules.

There is a "coupling spectrum" as illustrated in the diagram below:



There are various types of *Coupling* as discussed below:

[1] *No Direct Coupling*

These are independent modules and so are not really components of a single system.

[2] *Data Coupling*

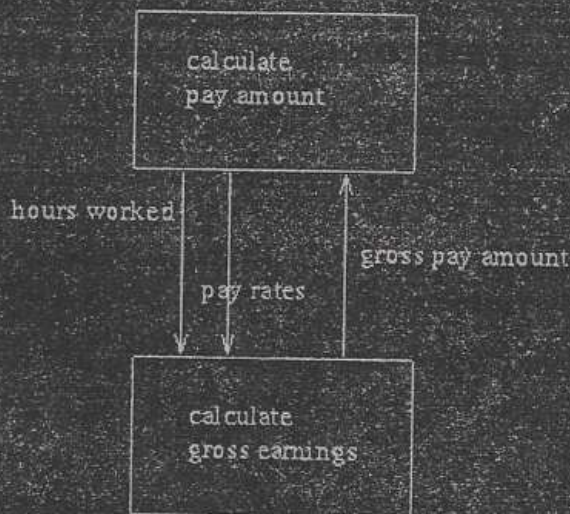
Two modules are data coupled if they communicate by passing parameters. This has been told to you as a "good design principle" since day one of programming instruction. See the diagram below.

data coupling

Two modules are data coupled if they communicate by passing parameters.

This has been told to you as a "good design principle" since day one of your programming instruction.

Example:



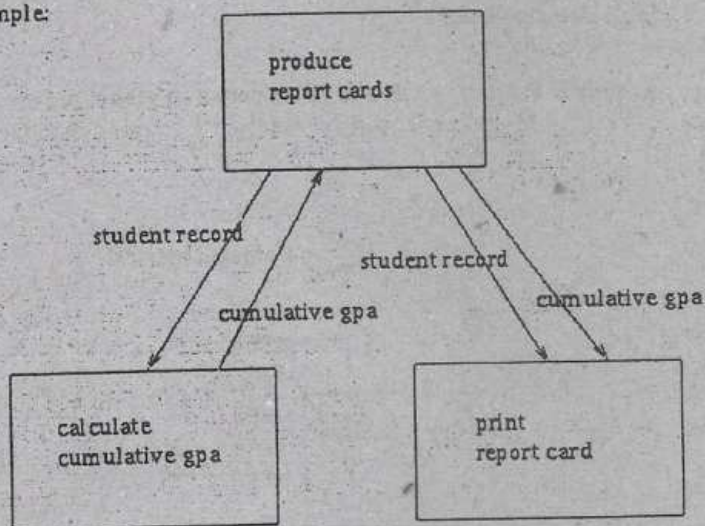
[3] *Stamp Coupling*

Two modules are stamp coupled if they communicate via a passed data structure that contains more information than necessary for them to perform their functions. See the diagram below.

stamp coupling

Two modules are stamp coupled if they communicate via a passed data structure which contains more information than necessary for the modules to perform their functions..

Example:



Here we assume the "student record" contains name, address, SSN, outside activities, medical information, contact names, etc... in addition to academic performance information.

[4] Control Coupling

Two modules are control coupled if they communicate using at least one "control flag". See the diagram on page 401.

[5] Common Coupling

Two modules are common coupled if they both share the same global data area. Another design principle you have been taught since day one: don't use global data. See the diagram titled 'Common Coupling' on the next page.

[6] Content Coupling

Two modules are content coupled if:

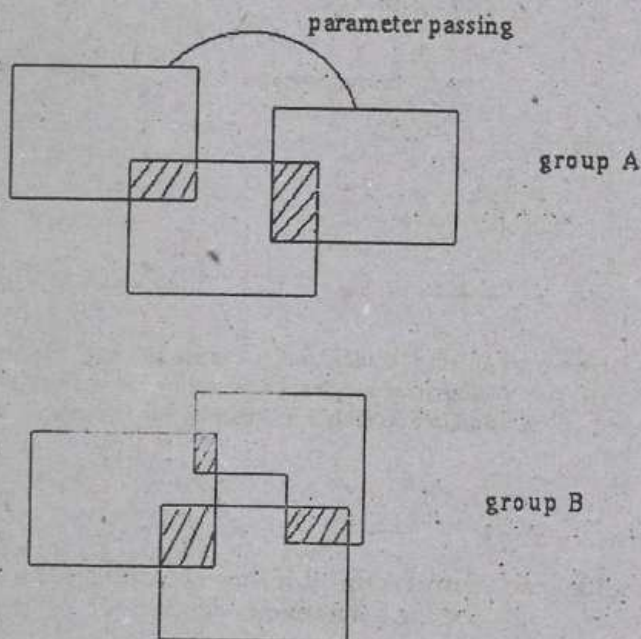
1. One module changes a statement in another (Lisp was famous for this ability)

common coupling

Two modules are common coupled if they both share the same global data area

Another design principle you have been taught since day-one: don't use global data

Example: Venn diagrams to show common global areas
Module group A is less tightly coupled than is group B



2. One module references or alters data contained inside another module
3. One module branches into another module

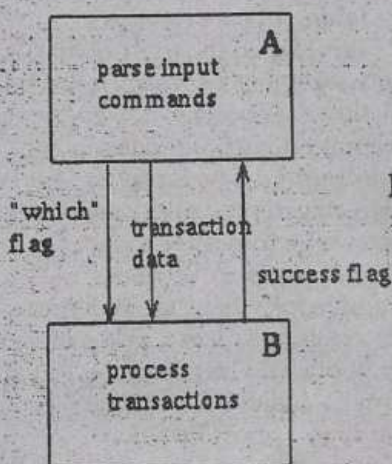
C.7.5 Summary: Coupling and Cohesion

In the case of "poorly cohesive programs", the information in the procedure may appear haphazard and unstructured to another

control coupling

Two modules are control coupled if they communicate using at least one "control flag"

Example:



A code:

```

{...
  call B(t_info, "update",ok)
  if not(ok) then ...
  ...
}
```

B code:

```

{receive parameters
  data, flag, outcome
  ...
  case (flag) of
    select: { ... }
    update: { ... }
    ...
    define: { ... }
    other: outcome := not ok
  end case
  outcome := ok
}
```

programmer, hence the program is harder to understand. In much the same way it is harder for us to understand what is going on in a picture the more the picture contains random and haphazard features. Driving home at night in a heavy rainstorm is harder than driving home on a clear, sunny day. An obscure or complicated job description is harder to understand than a succinct and precise job description.

In the case of "highly coupled programs/modules", the amount of information in the program might be too high, as in the case of long parameter lists. Or, some information may not be immediately visible to the programmer, as in the case of global variables and tables.

If you think about programming in terms of information that is immediately and easily discernible to the programmer, it becomes clear that programs occupy "space" in programmers' minds. The larger and more chaotic that space, the more difficult it is to understand the program!

Thus, to summarize:

The main goal of design (from a low-level perspective) is to "minimize COUPLING and maximize COHESION". Coupling is the level of

interdependency between a method and the environment, (other methods, objects and classes), while cohesion is the level of uniformity of the method goal. Interestingly, while coupling needs a low-level perspective, cohesion needs a higher point of view.

C.8 Six Sigma

C.8.1 Introduction

These days, a lot of discussion is taking place on this topic. In this section, we wish to provide a brief overview on the topic of Six Sigma. Six Sigma is today's most dynamic management approach for dramatically improving *product and process quality*. Its success, however, required a considerable amount of commitment on part of management.

Six Sigma is a management philosophy developed by Motorola that emphasizes setting extremely high objectives, collecting data, and analyzing results to a fine degree as a way to reduce defects in products and services. The Greek letter sigma (σ) is sometimes used to denote variation from a standard. The philosophy behind Six Sigma is that if you measure how many defects are in a process, you can figure out how to systematically eliminate them and get as close to perfection as possible. In order for a company to achieve Six Sigma, it cannot produce more than 3.4 defects per million opportunities.

Thus, the term "Six Sigma" is a statistical term that refers to 3.4 defects per million opportunities (or 99.99966 percent accuracy), which is as close as anyone is likely to get to perfect. A defect can be anything from a faulty part to an incorrect customer bill. Six Sigma teams use extremely rigorous data collection and statistical analysis to ferret out sources of errors and to find ways to eliminate them.

There are two Six Sigma processes: *Six Sigma DMAIC* (Define, Measure, Analyze, Improve, Control) and *Six Sigma DMADV*, each term derived from the major steps in the process. Six Sigma DMAIC is a process that defines, measures, analyzes, improves, and controls existing processes that fall below the Six Sigma specification. Six Sigma DMADV (Define, Measure, Analyze, Design and Verify) defines, measures, analyzes, designs, and verifies new processes or products that are trying to achieve Six Sigma quality. All Six Sigma processes are executed by Six Sigma Green Belts or Six Sigma Black Belts, which are then overseen by a Six Sigma Master Black Belts, terms created by Motorola.

C.8.2 What is Six Sigma in the Statistical Context

"Six Sigma" is a statistical concept that measures a process in terms of defects. Achieving Six Sigma means that your processes are delivering on 3.4 defects per million opportunities (DPMO). In other words, they

are working nearly perfectly. Sigma (the Greek letter σ) is a term in statistics that measures something called "standard deviation". In business context, it indicates defects in output of a process, and helps us to understand how far the process deviates from perfection. A Sigma represents 691462.5 defects per million opportunities, which translates to a percentage of non-defective outputs of 30.854%. This shows bad performance. If we have processes functioning at a three sigma level, this means we are allowing 66807.2 errors per million opportunities or delivering 93.319% non-defective production.

The central theme in Six Sigma Management is that if you can measure the defects in a process, you can systematically figure out ways to eliminate them, approach a quality level of zero defects. "Defect" is a measurable characteristics of the process or its output that is not within the acceptable customer limits i.e. not conforming to specifications. Six Sigma is about practices that help you eliminate defects and *always* deliver products and services that meet customer expectations.

So, in short, Six Sigma is several things:

- A statistical basis of measurement: 3.4 defects per million opportunities
- A philosophy and a goal: being as perfect as practically possible
- A methodology towards achieving excellence
- A symbol of quality

C.8.3 How does Six Sigma work?

Measurement & Metrics (The topic of Chapter 5) lie at the heart of Six Sigma. The basic approach is to measure performance on an existing process, compare it with a statistically valid ideal and figure out how to eliminate any variation. Project teams might speak in terms of reducing cycle time, improving customer satisfaction, cutting down on returns and improving the speed and accuracy of order fulfillment. No project is considered complete until the benefit has been shown and a team of financial auditors signs off.

In essence, DMAIC is the Six Sigma Improvement Framework There are five steps in DMAIC:

1. **Define:** A Six Sigma project team—led by a black belt—identifies a project based on business objectives as well as customer needs and feedback. The team identifies CTQs (critical to quality characteristics) that the customer considers to have the most impact on quality. It also separates the "vital few" from the "trivial many" (the projects that will have the most impact versus those that could stand improvement but are not critical).
2. **Measure:** The team identifies the key internal processes that influence critical process parameters impacting quality (the CTQs) and measures the defects currently generated relative to those processes.

3. **Analyze:** The team discovers why defects are generated by identifying the key variables that are most likely to create process variation.
4. **Improve:** The team identifies the maximum acceptable ranges of the key variables and validates a system for measuring deviations of the variables. The team modifies the process to stay within the acceptable range.
5. **Control:** Tools are put in place to ensure that the key variables remain within the maximum acceptable ranges over time.

C.8.4 Six Sigma: A Panacea for All Problems?

Six Sigma proponents claim that its benefits include up to 50% process cost reduction, cycle-time improvement, less waste of materials, a better understanding of customer requirements, increased customer satisfaction, and more reliable products and services. It is acknowledged that Six Sigma can be costly to implement and can take several years before a company begins to see bottom-line results. Motorola Texas Instruments, Scientific-Atlantic, General Electric, and Allied Signal are a few of the companies that practice Six Sigma.

If you are already operationally excellent and want to squeeze out defects and increase efficiency, Six Sigma is for you. If your business is built on intangibles—those aspects that are awfully hard to subject to statistical analysis—then Six Sigma is not for you. If you want to be a product leader competing primarily on quality—but not first-to-market—then do go for Six Sigma.

A word of caution: Data collection is the Achilles' heel of many a Six Sigma effort. This sort of woe, often caused by an amalgamation of disparate computer platforms thrown together in a merger or acquisition, is common at the large companies (several billions in revenues) that have been early adopters of Six Sigma. Making matters worse, to the best of author's knowledge, there is currently no software program that goes across a variety of computing platforms to collect data for Six Sigma analysis.

REFERENCES

- [1] *Statistical Quality Control*—Eugene L. Grant and Richard S. Leavenworth (Tata McGraw Hill)
- [2] *Measuring the Software Process*—William A. Florac and Anita D. Carleton (Addison Wesley)
- [3] *Statistical Quality Assurance Methods for Engineers*—Stephen B. Vardeman, J. Marcus Jobe ISBN: 0-471-15937-9
- [4] *The Practical Guide to Structured Systems Design*—by Meilir Page-Jones (Prentice Hall International Edition)
- [5] *Software Engineering: Planning for Change*—by David Alex Lamb (Prentice Hall International Edition)

- [6] *Experiments with Coupling and Cohesion Metrics in a Large System*—by Timothy C. Lethbridge and Nicolas Anquetil (School of Information Technology and Engineering University of Ottawa)
- [7] *Requirements Engineering: A Good Practice Guide*—Ian Sommerville and Pete Sawyer (John Wiley & Sons)
- [8] *Software Requirements and Estimation*—Swapna Kishore and Rajesh Naik (Tata McGraw Hill)
- [9] *Managing the Requirements Process*—Suzanne Robertson and James Robertson (Addison Wesley)
- [10] *Software Requirements*—Karl E. Wiegers (Microsoft Press)
- [11] *Object Oriented Modeling and Design*—James Rumbaugh (Prentice Hall India Ltd)
- [12] *The Object-Oriented Through Process*—Matt Weisfeld (Techmedia)
- [13] *Object Oriented Software Engineering*—Ivar Jacobson (Addison Wesley)
- [14] *Object Oriented Analysis and Design*—Grady Booch (Addison Wesley)
- [15] *Object-Oriented Design Measurement*—Scott A. Whimire (John Wiley & Sons Inc)
- [16] *Legal Issues in e-Business*—Nina Godbole (December 2001 and January 2002 issues of the Information Technology Magazine: A Delhi-based Publication)
- [17] *What Is Six Sigma?*—Peter S. Pande, Larry Holpp, Pete Pande, Lawrence Holpp (Tata McGraw Hill)

Index

A

- Acceptance criteria, 271
- Acceptance test, 37, 41, 46, 55, 267, 276, 277
- Acquiring test tools, 297
- Actual Cost of Work Performed (ACWP), 138
- ACWP (see Actual Cost of Work Performed), 138
- Adaptive maintenance, 367
- Adequacy audit, 38
- Alpha testing, 277
- ANSI Standard (American National Standard)—for reviews, 60
- Anti-pattern, 198
- Application architecture, 62
- Application granularity, 149
- Application under test (AUT), 269
- Appraisal costs, 5
- ARC (see Assessment Requirement for CMM-I), 339
- ASQ (American Society for Quality)
- ASQ Certification Program—history, 323
- Assessment
 - classes of, 339
 - definition, 210
 - first party, second party, third party, 211
 - method Documentation, 340
 - of software production process, 92
 - process of, 213
 - requirement for CMM-I (ARC), 339
 - types of, 211
- Assessments/Audits—preparation, 212
- Attribute Control Charts, 365
- Audit

- classification of (British Standards Institute), 211
- definition (IEEE), 210
- first party, second party, third party, 212
- trail, 189
- types of, 211
- Auditing, 44
- Audits and Assessments, 210
 - gap identification through, 213
- AUT—see application under test, 269
- Author
 - in inspection process, 158
 - in review process, 161
- Automated test packages, 317
- Automated test tools—selection process, 298
- Awareness Programs, 334

B

- Backlog Management Index (BMI), 112
- Bad fixes, 109
- Baselines, 179
 - IEEE definition, 180
- BCWP (see Budgeted Cost of Work Performed), 138
- BCWS (see Budgeted Cost of Work Scheduled)
- Beta testing, 277, 278
- Bi-directional Traceability, 61, 387
 - for requirements, 115
- Black box testing, 273, 293, 295
- BMI (See Backlog Management Index)
- Boehm—view of software product quality, 85
- Boundary value analysis, 281
- Brief history of ISO 9001, 200

British Standards Institute—audit classification, 211
Budgeted Cost of Work Performed (BCWP), 138
Budgeted Cost of Work Scheduled (BCWS), 138
Bugs v/s Defects, 106
Bugs, 106
Burn out problem—software industry, 357
Business Excellence Award of Europe, 247
Business value—ISO certificate, 208

C

Canada Award of Excellence (CAE), 247, 248
Capability determination—through assessment, 211
Capability Evaluation, 339
Capability Maturity Model
 SW CMM, 93, 224
 Integrated CMM-I, 235
 types, 234
CAR (see Causal Analysis and Resolution)
Career objectives, 311
Career paths—for quality professionals, 258, 311
Carnegie Mellon University, 93
Causal Analysis and Resolution (CAR), 72
Causal Analysis, 62
CCB (see Change Control Board, Configuration Control Board)
Certification
 and Assessments, 211
 benefits and impact, 321
 from QAI, 323
 ISO, 201
Change control—in testing context, 282
Change Control Board (CCB), 184
Change log, 184
Change management, 173
Change Request—Format, 187
Checklist for evaluation—SCM Plan, 196
Chidamber & Kemerer's Metrics suite, 149

Choosing a CMM-I representation—factors to consider, 243
Clause 8.2.2 of ISO 9001:2000, 209
CMM (see Capability Maturity Model)
CMM and SCM, 193
CMM-based Software Process Improvement, 224
CMM I (Integrated Capability Maturity Model), 235
 Background of, 236
 choosing a representation for, 243
 Components of, 236, 238
 disciplines, 236, 237
 generic practices in, 242
 maturity levels, 239, 241
 model evolution, 236, 237
 myths & realities, 237, 338
 process areas, 239
 process area grouping, 241
 process areas selection, 339
 representations of, 236, 237
 representation selection—factors to consider —, 243
 specific practices, 242
 staged v/s continuous representation, 242
COCOMO (See Constructive Cost Model)
Code control, 63, 173
Code walkthroughs, 158
Coding guidelines, 94
Cognitive Dissonance, 336
 effect on reviews & inspections, 166
Cohesion, 389, 390
 as a design principle, 11, 45
 types of, 391
Common Features—SW CMM, 94
Communication skills—for quality professionals, 314
Comparison b/w SW CMM and CMM-I, 244
Comparison between SW CMM and ISO, 227, 228
Comparison of SW CMM Model with ISO Models, 224
Complexity
 apparent, 382
 inherent, 382
 measure for, 18
 metrics for, 18

- Compliance audit, 212
 - Components of the CMM-I Model, 236, 238
 - Configuration Audit
 - guidelines for, 191
 - Configuration, 174, 176
 - board for (Configuration Control Board—CCB), 184
 - control, 63, 184
 - controller, 184
 - identification of, 63
 - in testing context, 282
 - items under, 64
 - management—ISO Clauses for, 192
 - management of, 41, 63, 174
 - management—plan for, 59
 - management—why, 174
 - process for management of, 176
 - status Accounting, 63, 183
 - v/s Document Control, 179
 - Conflict management—in reviews, 160
 - Constraint, 10
 - Constructive Cost Model (COCOMO), 115
 - Constructive Quality Model (COQUAMO), 88
 - Consulting services—ISO, 216
 - Continual improvement, 30
 - Continuous (process) improvement, 29, 80, 318
 - Continuous Representation—Integrated CMM, 239, 242
 - Contractual document, 37
 - Control Charts, 99, 361, 362, 366
 - limitations of, 365
 - purpose of, 362
 - types of, 365
 - Control Flow Graphs, 385
 - Control Limits—Upper and Lower, 362, 363
 - Control of Documents and records—ISO 9001:2000 clause, 66
 - Conventions (IEEE Std. 983 definition), 83
 - Corporate audits, 317
 - Corporate objectives—quality, 318
 - Corrective action, 63
 - Corrective maintenance, 331, 366
 - Correctness, 84
 - Correspondent Members of ISO 9001, 201
 - Cost
 - Estimate of, 42
 - of (poor) quality, 162, 259
 - Performance Index (Cost Performance Index—CPI), 138
 - tracking of, 42
 - variance of (cost variance—CV), 138
 - Costing technique, 42
 - Country subscriber—membership for ISO 9001, 201
 - Coupling, 11, 388, 395
 - types of, 397
 - Coupling & Cohesion, 11, 33, 62, 388
 - summary of, 400
 - CSTE (Certified Software Test Engineer)
 - CSTE program from QAI, 308
 - Cultural differences—maturity (supplier and customer), 190
 - Cultural issues in Reviews, 165
 - Customer-reported bugs and defects, 71, 108
 - Cyclomatic complexity, 11, 146, 291, 382
 - for maintenance implications, 383
 - implications for testing, 384
- D**
- Database integrity testing, 315
 - Debugging Process, 263
 - Debugging v/s Testing, 262
 - Decision Analysis & Resolution (DAR), 44
 - Decision making rationale, 44
 - Defect
 - density v/s MTTF, 107
 - density, 106
 - injection rate, 108
 - levels & user satisfaction—the relationship, 105
 - measures and metrics, 105, 106
 - prevention activities for, 71, 104, 105
 - rate of, 106
 - removal effectiveness, 108, 109
 - removal efficiency, 108, 114

- severity, 71
- tracking report, 113
- v/s bug, 106
- Defective fix, 113
- Defects—customer reported, 108
- Defined software process, 67
- Degree of cohesion of objects, 147
- Degree of reuse—inheritance methods, 148
- Delinquent fixes, 113
- Delta report, 185
- Design Review, 61
- Design validation, 40
- Desk investigation, 214
- Desk-checks, 155, 168
- Development Stages of ISO 9001, 203
- Disaster Recovery Plan (DRP), 64
- Document Control
 - ISO expectations, 179
 - v/s Configuration Control, 179
- Document of understanding (DoU), 37

E

- Earned Value (EV), 138
- Earned Value Analysis (EVA), 34, 35, 136, 137
- Earned Value Management (EVM), 137
- e-Business
 - and ISO, 222
 - models for, 372
 - websites—perfective maintenance, 379
- Edwards Deming, 93
- Effective inspection, 166
- Effective Test Case, 260
- Efficiency, 84
- Effort as a measure, 136
- EFQM—fundamental concepts, 250, 251
- Engineering Maturity Model (EMM), 252
- Enhancement projects, 102
- Entity Life History Diagram, 62
- Entity Relationship Diagram (ERD), 62
- Equivalence partitioning, 279, 280, 281
- Errors, 106

- Estimates, 38
- EVA (see Earned Value Analysis)
- EVA
 - an illustrative example, 139, 140, 141
 - benefits to project management, 139
 - success essentials for, 141
 - ten commandments of, 141
- EVM (see Earned Value Management)
- Evolution of software systems, 81
- Evolutionary Approach to Software Product Development, 88
- External assessors/auditors—ISO, 210
- External attributes—software product, 83
- External audits, 49
- External motivators, 311
- Extreme programming, 300
- Extreme testing, 300

F

- Factoring effectiveness, 148
- Failure costs, 5
- Failures, 106
- Fan-in and Fan-out, 291
- Fault recovery testing, 285
- Final/External assessments/audits—ISO, 210
- First-party-assessment and audit, 211, 212
- Fish Bone Diagram, 72
- Fix Backlog, 112
- Fix quality, 113
- Fix-response-time, 113
- Form attributes, 84
- Formal inspection, 168
- Formal Notation for SRS, 18
- Formal Technical Review (FTR), 155, 159
- Format for Change Request, 187
- Function points and feature points, 103
- Functional attributes, 84
- Functional Audit, 62
- Functional objectives, 88
- Functional requirements, 116
- Functional testing techniques, 279
- Functional traceability, 39

- Functionality, 83
 Function-oriented software metrics, 103
- G**
- Gap analysis and identification—
 Audits/Assessments, 213
 Gary box testing, 295
 Generic Practices in CMM-I, 242
 Goal, 10
 Goal-Question-Metric Model, 87
 Gray box testing, 273, 293
 Group Think Effect, 336
 GUI standards, 94
- H**
- High Maturity Organizations, 227, 325, 329
 general philosophy and the SEI Survey, 331
 Human issues and challenges in testing, 306
- I**
- IEC (See International Electro-Technical Commission), 202
 IEEE—SCM snapshot, 196
 IEEE—testing related definitions, 259, 270, 271
 IEEE (see Institute of Electrical and Electronic Engineers)
 IEEE 1074, 246
 audit—definition, 210
 baseline—definition, 180
 guide to (software) requirements specification, 61, 385
 maintenance model, 370
 quality metric definition, 88
 SCI—definition, 177
 SCM—definition, 175
 standard for software quality assurance plan, 56, 57
 validation—definition, 258
 IEEE Standards
 Std 1061, 89
 Std. 610.12, 82—software product definition, 82
 Std. 610.12, 82, 83
 Std. 983—for Conventions, 83
 Std 828:1990—for SCM Plans, 195
 Std 1042: 1987—Guide to SCM, 195
 Incremental and Evolutionary models, 368
 Independent SQA Group, 333
 Independent test observer, 266, 267, 268
 Independent test observer, 276
 Indian software scenario
 exports growth, 353
 NASCOM survey, 352
 people management problems & challenges, 353, 356
 quality maturity, 352
 software practices maturity, 341
 top twenty companies, 349
 Informed decision, 331
 Inheritance, 372
 dependencies, 147
 tree depth, 149
 In-Process Audit, 62
 In-Process-Defects, 108
 Inspection
 advantages/benefits of, 168, 169
 as a static testing technique, 261
 checklists, 169, 170, 171
 conflict management, 160
 reasons for effectiveness, 165
 typical problems, 166
 Inspection, 155, 158, 260
 responsibilities of the leader -, 159
 methods for, 166
 v/s Testing, 260
 yield, 166
 Installation testing, 284
 Institute of Electrical and Electronic Engineers (IEEE), 80
 Integrated Capability Maturity Model (CMM I), 235
 continuous representation, 239, 242
 disciplines for, 236, 237
 generic practices, 242
 maturity levels, 239, 241
 myths & realities, 237, 338
 Process Area grouping, 241
 Process Areas selection, 239, 339
 representations, 236, 237
 specific practices, 242
 staged representation, 239, 240
 vis-à-vis SW CMM, 235
 Integration testing, 42, 275

- Internal attributes of software product, 83
- Internal Audit, 49, 212, 214
- International Electro-Technical Commission (IEC), 202
- International Organization for Standardization (ISO), 228, 231
- Inter-object coupling, 147
- Intrusive testing, 281
- ISO (International Organization for Standardization), 228
 - ISO
 - brief history, 200
 - central secretariat, 202
 - document control, 179
 - FAQs (Frequently Asked Questions), 221
 - MoU for support to e-Business, 222
 - standards—development process, 203
 - ISO 12207, 246
 - organizational management processes, 246
 - primary processes, 246
 - supporting processes, 246
 - ISO 8402
 - definition of software quality, 88
 - ISO 9000
 - need for, 207
 - ISO 9000:1987
 - first revision to, 205
 - ISO 9000:1994, 205
 - clauses, 205
 - major and minor changes, 206
 - ISO 9000-3, 90, 92
 - ISO 9001, 92, 200
 - audits/assessments, 200
 - continual improvement context, 200
 - correspondent members, 201
 - country subscriber membership, 201
 - Final Draft International Standard (FDIS), 204
 - member body, 201
 - member constitution, 200, 201
 - O Members and P Members, 201
 - origins of, 201, 201
 - structure of, 200, 201
 - working of, 200, 205
 - Draft International Standard (DIS), 204
 - use of consultants, 200
 - ISO 9001, 9002, 9003, 205
 - ISO 9001:2000
 - clause 8.2.2, 209
 - clause about control of documents and records, 66
 - clause about suppliers, 65
 - ISO 9004-1, 205
 - ISO 9126
 - quality definition, 87
 - standard quality model, 87
 - ISO
 - and e-Business, 222
 - certificate as a business reference, 208
 - certification, 208
 - configuration management related clauses, 192
 - definition of audit/assessment, 210
 - mapping to SW CMM, 228
 - ISO consultants
 - handholding with the client, 221
 - involvement phases, 220
 - selection and usage, 213, 215, 217, 218, 219
 - services offered, 215, 216
 - ISO memento, 203
 - ISO registrars, 208
 - ISO technical committee, 203
 - ISO/IEC 15504, 231, 232
 - ITC (See ISO Technical Committee), 203
- J
 - Juran, 93
- K
 - KPA (see Key Process Areas)
 - Key Practices—SW CMM, 225
 - Key Process Areas (KPA)—SW CMM, 94, 225, 234
- L
 - Labels—in configuration management, 179
 - Lessons learnt, 38, 327

- Levels of control—SCM, 182
- Library function, 63, 64
- Life Cycle
 - testing, 264
 - website, 375, 376
- Limitations of Control Chart—365
- Linear Sequential Approach to Software Development, 88
- Load testing, 288
- Low morale & burnout, 312
- M**
- Maintainability—object orientation context, 371
- Maintainability, 83, 85
- Maintenance
 - adaptive, 367
 - corrective, 366, 371
 - perfective, 367
 - of website, 373
- Maintenance models (for software)
 - evolution, 367, 369
 - IEEE, 370
- Maintenance Projects—Project Management Issues, 331
- Major and Minor changes to ISO 9000:1994, 206
- Major NC, 214
- Malcolm Baldrige National Quality Award (MBNQA), 247
- Management commitment, 30
- Management reviews, 62
- Managing web-site maintenance, 377, 378
- Mapping
 - ISO 9001 and CMM, 228
 - SW CMM and ISO/IEC 15504, 232
- Maturity, 81, 224
- Maturity levels, 93
 - Integrated CMM, 239, 241
 - P-CMM, 254
 - SW CMM, 225
- Mature organizations—practices followed in, 328, 329
- Maturity model, 89
- MBNQA (see Malcolm Baldrige National Quality Award)
 - as TQM framework, 247
 - equivalent awards in other countries, 247
- McCabe's cyclomatic complexity number, 291
- McCall
 - software product quality model, 85, 86
 - factor-criteria-metric Model, 85
- Mean time to failure (MTTF), 106
- Measurable objectives, 88
- Measurement
 - definition, 97
 - in High Maturity Organizations, 333
 - in project, 135, 136
 - need for, 98
 - principles and protocols, 120, 123, 124
 - scales, 122
 - steps, 99
 - structural model for, 121
 - theory, 96
 - user satisfaction context, 105
- Measures v/s metrics, 96
- Member Body of ISO 9001, 201
- Member constitution of ISO 9001, 200, 201
- Mentor, 319
 - role for quality professionals, 311
- Mentoring, 313, 318, 334
- Method complexity, 148
- Metrication program, 132
- Metrics
 - derived, 101
 - object oriented design, 151
 - object oriented software development, 146
 - requirements related, 115, 116
 - software maintenance, 110, 111
 - test coverage context, 292
 - test plan context, 292
 - test result context, 292
- Metrics program, 68, 70, 71
 - development—issues to be considered, 71
 - implementation—issues, 142, 143
 - planning, 141, 142
- Metrics-based proactive project tracking, 138
- Michael Fagan of IBM, 158
- Mini assessment process, 337
- Minor NC, 214

Mission statement, 25
 Models for product quality and process quality, 80, 85, 87
 Moderator
 basic tasks, 160
 in inspection process, 159
 Modifiability, 17
 MTTF (see Meant Time to Failure)
 MTTF v/s Defect Density, 107
 Multiple reviewers—importance, 166
 Myers—views on walkthroughs, 157

N

NASCOM survey—quality standards by Indian software and IT organizations, 352
 NC (see Non-conformities/Non-conformances)
 Non-conformances, 49
 major and minor, 214
 Non-functional requirements, 10, 284, 294, 386
 Non-functional testing techniques, 279, 283

O

Object classes—coupling, 150
 Object library effectiveness, 148
 Object Orientation and Maintainability—371
 Object oriented metrics: an overview, 146
 Objective evidence, 28
 Object-oriented analysis and design (OOAD), 146
 Object-oriented design, 11
 Objects—degree of cohesion, 147
 ODC (see orthogonal defect classification)
 O-Members of ISO 9001, 201
 On-site assessment process, 214
 Operational integrity, 84
 Origins of ISO 9001, 201, 201
 Orthogonal defect classification (ODC), 293
 Other process improvement models, 253
 Outsourcing, 64

P

PA (see Process Areas)
 Past performance data, 136
 illustration on its use, 100
 usefulness, 133
 Path coverage, 290
 P-CMM (see People CMM)
 People CMM (P-CMM), 224, 252, 254, 255
 aim of, 342
 maturity levels, 254
 scenario in India, 342, 343
 model, 312, 329, 330
 People issues
 in quality assurance, 53, 311, 312
 in management problems—Indian software industry, 356
 People maturity, 224
 Perfective Maintenance, 367
 e-Business websites, 379
 Performance attributes, 84
 Performance testing—285, 286
 Personal Software Process (PSP), 336
 Philip Crosby, 93
 Pitfalls in Software Configuration Management, 197
 P-Members of ISO 9001, 201
 Portability, 84
 Practices at mature organizations, 224
 Pre-assessment, 213
 Preparing for assessments/audit, 212
 Prevention costs, 5
 Primary metrics, 333
 Priority number, 63
 Problems v/s failures, 106
 Process
 capability baselines (PCB), 133
 change management (PCM), 74, 76
 change request for (PCR)—Format, 76
 champions for, 335
 consulting for, 311
 definition of, 325
 important aspects of, 325, 326
 management of, 67
 maturity of, 326
 metrics, 11, 114, 135
 quality, 92, 126
 variability, 361